

SYDE 572 — Assignment 1

Shortest distance from a point to a function; fitting a line and a parabola

September 23, 2026

Project webpage: <https://Todoro0-0.github.io/1/>

Contents

1	Shortest distance from a point to a function	1
1.1	Objective function	2
1.2	Analytical solution	2
1.3	Numerical method A: Newton–Raphson	3
1.4	Numerical method B: golden section search	5
1.5	Results for the assigned points	6
1.6	Python implementation	7
1.7	Other parabolas and non-polynomial functions	7
2	Fitting a line and a parabola	10
2.1	Setting up the MSE	10
2.2	Analytical solution	10
2.3	Numerical solution: Newton–Raphson, one parameter per step	11
2.4	Results	13
2.5	Python implementation	14
A	Intermediate-step plots for the remaining points	15
B	distance.py — Part 1 code	18
C	fitting.py — Part 2 code	30

1 Shortest distance from a point to a function

Problem. Solve analytically and with two numerical approaches (Newton–Raphson, golden section search) for the shortest distance from $(0, 0)$, $(-4, 0)$, $(-8, 0)$, $(2, 0)$, $(6, 0)$ to $y = x^2 + 5$. Code the numerical methods for any point and any parabola, demonstrate them on other functions and points, extend them to non-polynomial functions, and plot the intermediate steps.

1.1 Objective function

The distance from (x_0, y_0) to a point $(x, f(x))$ on the curve is

$$d(x) = \sqrt{(x - x_0)^2 + (f(x) - y_0)^2}. \quad (1)$$

The square root is monotonically increasing, so the minimiser of d also minimises the squared distance, which is what every method below works with:

$$D(x) = (x - x_0)^2 + (f(x) - y_0)^2, \quad (2)$$

$$D'(x) = 2(x - x_0) + 2(f(x) - y_0)f'(x), \quad (3)$$

$$D''(x) = 2 + 2f'(x)^2 + 2(f(x) - y_0)f''(x). \quad (4)$$

Setting $D'(x) = 0$ gives

$$(x - x_0) + (f(x) - y_0)f'(x) = 0 \iff (x - x_0, f(x) - y_0) \cdot (1, f'(x)) = 0, \quad (5)$$

so the segment from the point to the curve is perpendicular to the tangent at the closest point.

For $f(x) = x^2 + 5$ (hence $f' = 2x$, $f'' = 2$) with $y_0 = 0$:

$$D(x) = (x - x_0)^2 + (x^2 + 5)^2, \quad D'(x) = 4x^3 + 22x - 2x_0, \quad D''(x) = 12x^2 + 22. \quad (6)$$

$D''(x) > 0$ everywhere, so D is strictly convex: one stationary point, and it is the global minimum. All three methods must therefore agree.

1.2 Analytical solution

Dividing $D'(x) = 0$ by 4 gives the depressed cubic

$$x^3 + px + q = 0, \quad p = \frac{11}{2}, \quad q = -\frac{x_0}{2}, \quad (7)$$

whose discriminant

$$\Delta = \left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3 = \frac{x_0^2}{16} + \frac{1331}{216}, \quad \frac{1331}{216} = \left(\frac{11}{6}\right)^3 = 6.162037, \quad (8)$$

is positive for every x_0 . Exactly one real root exists, given by Cardano's formula:

$$x^* = u + v, \quad u = \sqrt[3]{\frac{x_0}{4} + \sqrt{\Delta}}, \quad v = \sqrt[3]{\frac{x_0}{4} - \sqrt{\Delta}}, \quad d = \sqrt{(x^* - x_0)^2 + ((x^*)^2 + 5)^2}. \quad (9)$$

Worked in full: the point $(-4, 0)$.

$$\frac{x_0}{4} = -1, \quad \sqrt{\Delta} = \sqrt{1 + 6.162037} = 2.676198, \quad (10)$$

$$u = \sqrt[3]{-1 + 2.676198} = \sqrt[3]{1.676198} = 1.187887, \quad (11)$$

$$v = \sqrt[3]{-1 - 2.676198} = \sqrt[3]{-3.676198} = -1.543357, \quad (12)$$

$$x^* = 1.187887 - 1.543357 = -0.355470, \quad y^* = (-0.355470)^2 + 5 = 5.126359, \quad (13)$$

$$d = \sqrt{(-0.355470 + 4)^2 + (5.126359)^2} = \sqrt{13.282599 + 26.279556} = \sqrt{39.562155} = \mathbf{6.289845}. \quad (14)$$

Check in (6): $4(-0.355470)^3 + 22(-0.355470) + 8 = -0.179668 - 7.820340 + 8 \approx 0$. ✓

All five points. Each row applies (9) in the same three steps.

Table 1: Analytical solution for $y = x^2 + 5$.

(x_0, y_0)	$x_0/4$	$\sqrt{\Delta}$	u	v	$x^* = u + v$	y^*	$D(x^*)$	d
(0, 0)	0.00	2.482345	1.354006	-1.354006	0.000000	5.000000	25.000000	5.000000
(-4, 0)	-1.00	2.676198	1.187887	-1.543357	-0.355470	5.126359	39.562155	6.289845
(-8, 0)	-2.00	3.187795	1.059044	-1.731122	-0.672078	5.451689	83.419352	9.133420
(2, 0)	0.50	2.532200	1.447391	-1.266647	0.180745	5.032669	28.637444	5.351396
(6, 0)	1.50	2.900351	1.638686	-1.118782	0.519904	5.270300	57.807516	7.603125

For (0, 0) the cube roots cancel ($u = -v$), so $x^* = 0$ and the closest point is the vertex (0, 5).

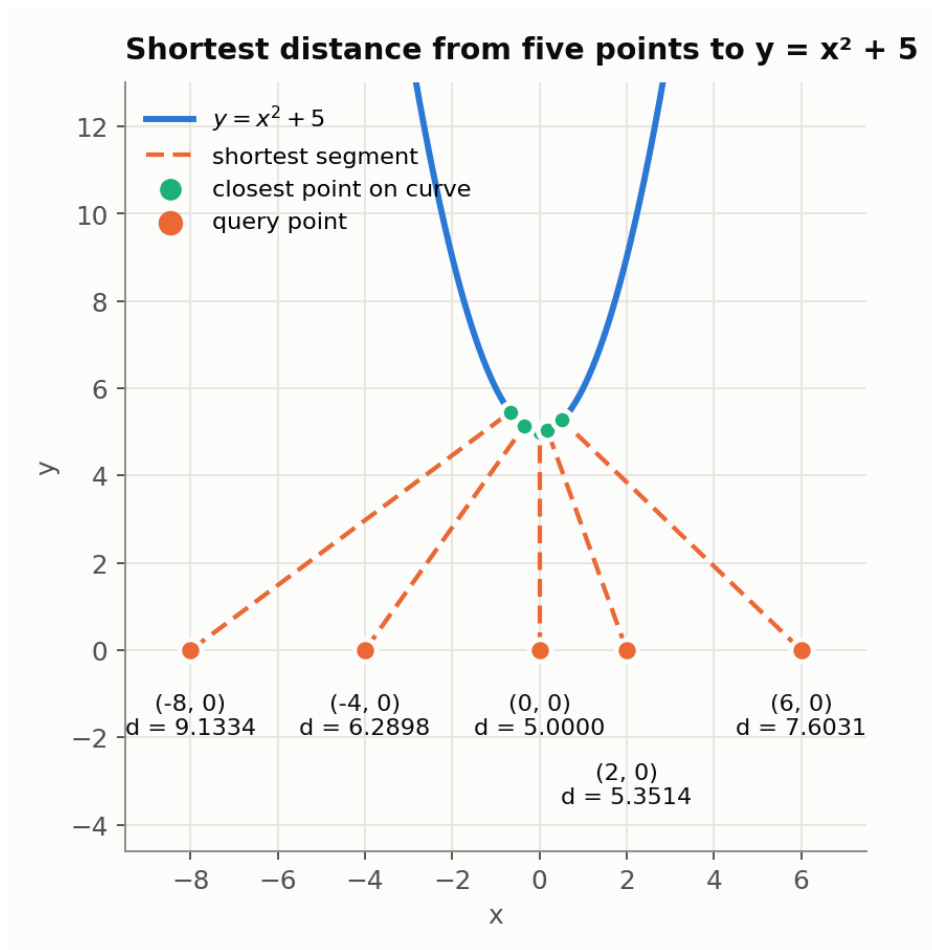


Figure 1: The five points, their closest points on $y = x^2 + 5$ and the shortest segments. Axes are equally scaled, so the right angles are visible.

1.3 Numerical method A: Newton–Raphson

Newton’s method is applied to the root of D' :

$$x_{k+1} = x_k - \frac{D'(x_k)}{D''(x_k)} = x_k - \frac{2(x_k - x_0) + 2(f(x_k) - y_0)f'(x_k)}{2 + 2f'(x_k)^2 + 2(f(x_k) - y_0)f''(x_k)}, \quad (15)$$

which for $f(x) = x^2 + 5$, $y_0 = 0$ simplifies to

$$x_{k+1} = x_k - \frac{4x_k^3 + 22x_k - 2x_0}{12x_k^2 + 22} = \frac{8x_k^3 + 2x_0}{12x_k^2 + 22} = \boxed{\frac{4x_k^3 + x_0}{6x_k^2 + 11}} \quad (16)$$

Every run starts at $x^{(0)} = 0$ with tolerance 10^{-7} , so the first step is always $x_1 = x_0/11$.

Worked in full: the point $(-4, 0)$.

$$x_1 = \frac{4(0)^3 - 4}{6(0)^2 + 11} = \frac{-4}{11} = -0.3636364, \quad (17)$$

$$x_2 = \frac{4(-0.3636364)^3 - 4}{6(-0.3636364)^2 + 11} = \frac{-0.1923366 - 4}{11.7933884} = -0.3554819, \quad (18)$$

$$x_3 = \frac{4(-0.3554819)^3 - 4}{6(-0.3554819)^2 + 11} = \frac{-0.1796676 - 4}{11.7581771} = -0.3554697, \quad (19)$$

$$x_4 = -0.3554697 \Rightarrow |x_4 - x_3| < 10^{-7}, \text{ stop}, \quad (20)$$

giving $d = 6.289845$. The number of correct digits roughly doubles per step ($2 \rightarrow 5 \rightarrow 10$): quadratic convergence.

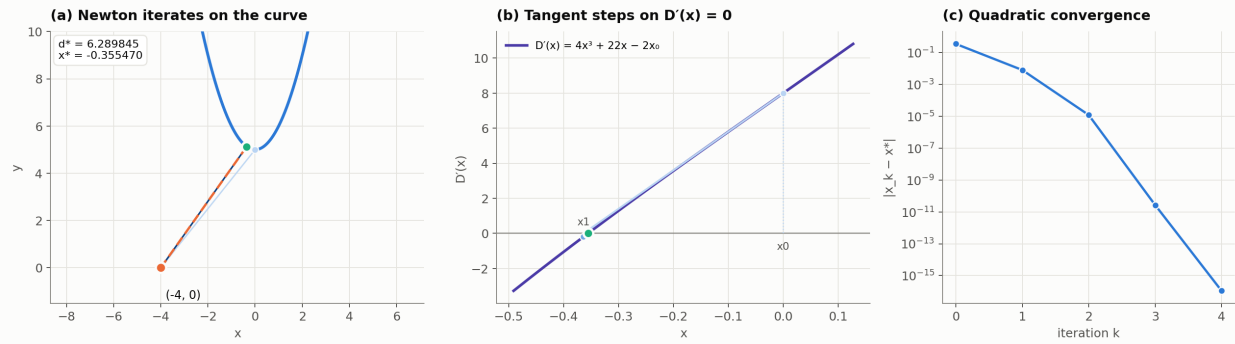
Table 2: Newton–Raphson for the point $(-4, 0)$, all quantities from (6).

k	x_k	$D(x_k)$	$D'(x_k)$	$D''(x_k)$	x_{k+1}	$ x_{k+1} - x_k $
0	0.0000000	41.000000	8.000000	22.000000	-0.3636364	3.64×10^{-1}
1	-0.3636364	39.562940	-0.192337	23.586777	-0.3554819	8.15×10^{-3}
2	-0.3554819	39.562155	-0.000288	23.516409	-0.3554697	1.22×10^{-5}
3	-0.3554697	39.562155	0.000000	23.516304	-0.3554697	2.72×10^{-11}

Table 3: Newton iterates for all five points, from the guess $x = 0$. Bold entries are converged. $(0, 0)$ needs no iteration because $D'(0) = 0$.

k	x_k for the query point (x_0, y_0)				
	$(0, 0)$	$(-4, 0)$	$(-8, 0)$	$(2, 0)$	$(6, 0)$
0	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000
1	0.0000000	-0.3636364	-0.7272727	0.1818182	0.5454545
2		-0.3554819	-0.6729923	0.1807447	0.5200682
3		-0.3554697	-0.6720784	0.1807446	0.5199037
4		-0.3554697	-0.6720781	0.1807446	0.5199037
5			-0.6720781		

Newton-Raphson · point (-4, 0) · initial guess $x = 0$



Newton-Raphson · point (-8, 0) · initial guess $x = 0$

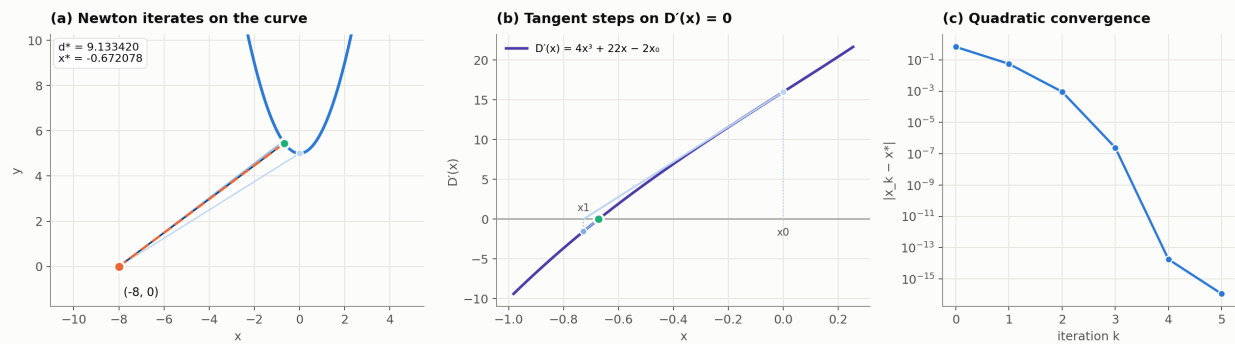


Figure 2: Newton-Raphson intermediate steps for $(-4, 0)$ and $(-8, 0)$. (a) The iterates on the curve, light $\rightarrow$ dark. (b) Each tangent to D' followed to its zero crossing gives the next iterate. (c) Error on a log scale. The other three points are in Appendix A.

1.4 Numerical method B: golden section search

The bracket $[a, b]$ carries two interior probes,

$$x_1 = a + r(b - a), \quad x_2 = b - r(b - a), \quad r = 2 - \varphi = \frac{3 - \sqrt{5}}{2} = 0.381966, \quad \varphi = 1.618034. \quad (21)$$

If $D(x_1) < D(x_2)$ the minimum of a unimodal D cannot lie in $(x_2, b]$, so $b \leftarrow x_2$; otherwise $a \leftarrow x_1$. The golden ratio places the surviving probe exactly where the new bracket needs one, so each iteration costs one new evaluation of D and shrinks the bracket by $1/\varphi = 0.618034$. No derivatives are used. Starting from $[-10, 10]$ with tolerance 10^{-7} ,

$$20\varphi^{-n} < 10^{-7} \implies n > \frac{\ln(20/10^{-7})}{\ln 1.618034} = \frac{19.1138}{0.4812} = 39.72 \implies n = 40 \quad (22)$$

iterations for every point.

Worked in full: the point $(-4, 0)$. At $x = \pm 2.3607$, $x^2 + 5 = 10.5729$, so

$$x_1 = -10 + 0.381966(20) = -2.3607, \quad x_2 = 10 - 0.381966(20) = 2.3607, \quad (23)$$

$$D(x_1) = (-2.3607 + 4)^2 + (10.5729)^2 = (1.6393)^2 + 111.786 = 114.47, \quad (24)$$

$$D(x_2) = (2.3607 + 4)^2 + (10.5729)^2 = (6.3607)^2 + 111.786 = 152.24. \quad (25)$$

Since $114.47 < 152.24$, $b \leftarrow x_2$ and the bracket becomes $[-10, 2.3607]$. The old x_1 becomes the new x_2 , so only $x_1 = -10 + 0.381966(12.3607) = -5.2786$ is evaluated, with $D = 1081.68$. Continuing:

Table 4: Golden section search for $(-4, 0)$ on the bracket $[-10, 10]$.

k	a	b	x_1	x_2	$D(x_1)$	$D(x_2)$	update	$b - a$
0	-10.0000	10.0000	-2.3607	2.3607	114.4717	152.2425	$b \leftarrow x_2$	20.0000
1	-10.0000	2.3607	-5.2786	-2.3607	1081.6804	114.4717	$a \leftarrow x_1$	12.3607
2	-5.2786	2.3607	-2.3607	-0.5573	114.4717	40.0544	$a \leftarrow x_1$	7.6393
3	-2.3607	2.3607	-0.5573	0.5573	40.0544	48.9709	$b \leftarrow x_2$	4.7214
4	-2.3607	0.5573	-1.2461	-0.5573	50.5232	40.0544	$a \leftarrow x_1$	2.9180
5	-1.2461	0.5573	-0.5573	-0.1316	40.0544	40.1382	$b \leftarrow x_2$	1.8034
6	-1.2461	-0.1316	-0.8204	-0.5573	42.2933	40.0544	$a \leftarrow x_1$	1.1146
7	-0.8204	-0.1316	-0.5573	-0.3947	40.0544	39.5803	$a \leftarrow x_1$	0.6888
$\vdots$ 32 further iterations $\vdots$								
40	-0.3554697	-0.3554696	stop: $x^* = \text{midpoint} = -0.3554697$, $d = 6.289845$					8.74×10^{-8}

The first probes are always ∓ 2.3607 , so for the other points only $(x - x_0)^2$ changes. The pair $(D(-2.3607), D(2.3607))$ is $(117.36, 117.36)$ for $(0, 0)$; $(143.59, 219.13)$ for $(-8, 0)$; $(130.80, 111.91)$ for $(2, 0)$; and $(181.69, 125.03)$ for $(6, 0)$. Each case takes 40 iterations.

Golden Section Search · point $(-4, 0)$ · bracket $[-10, 10]$

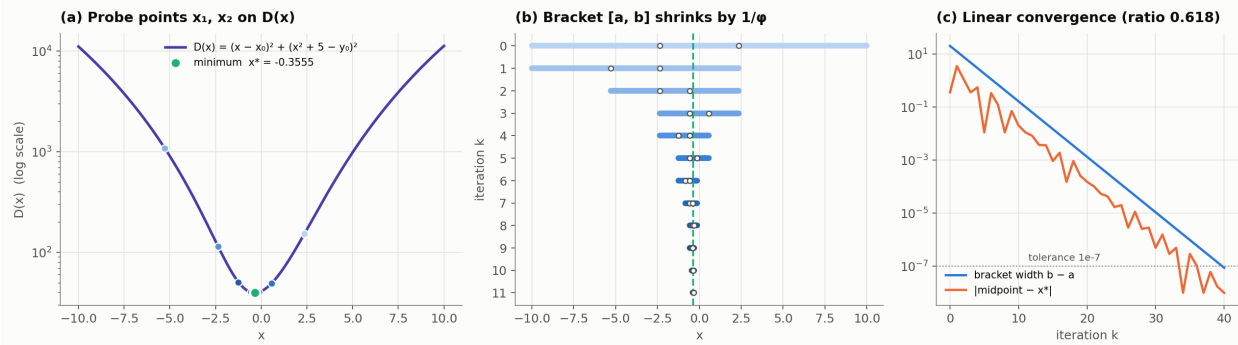


Figure 3: Golden section intermediate steps for $(-4, 0)$. (a) Probe pairs on $D(x)$. (b) The bracket for the first 12 iterations, white dots marking the probes. (c) Bracket width and midpoint error, both decaying by the factor 0.618 of (22).

1.5 Results for the assigned points

Table 5: The three methods agree to at least six decimal places.

(x_0, y_0)	Analytical		Newton–Raphson		Golden section		$\max \Delta d $
	x^*	d	d	iters	d	iters	
$(0, 0)$	0.000000	5.000000	5.000000	1	5.000000	40	8.9×10^{-16}
$(-4, 0)$	-0.355470	6.289845	6.289845	4	6.289845	40	1.8×10^{-15}
$(-8, 0)$	-0.672078	9.133420	9.133420	5	9.133420	40	0
$(2, 0)$	0.180745	5.351396	5.351396	4	5.351396	40	0
$(6, 0)$	0.519904	7.603125	7.603125	4	7.603125	40	8.9×10^{-16}

$d(0,0) = 5.000000$ at $x^* = 0$; $d(-4,0) = 6.289845$ at $x^* = -0.355470$; $d(-8,0) = 9.133420$ at $x^* = -0.672078$; $d(2,0) = 5.351396$ at $x^* = 0.180745$; $d(6,0) = 7.603125$ at $x^* = 0.519904$.

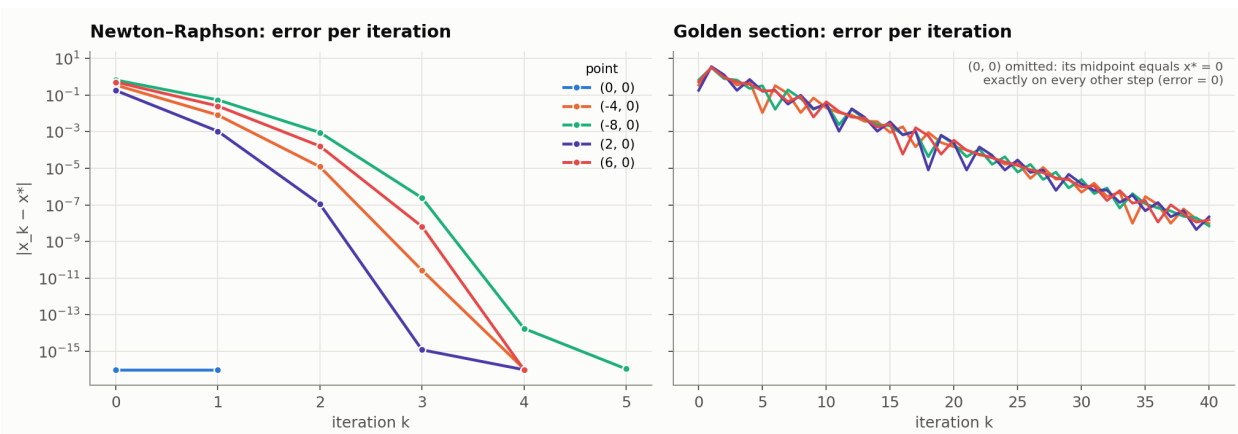


Figure 4: Error per iteration. Newton (left) converges quadratically; golden section (right) linearly, with slope $\log_{10} 0.618$ per iteration.

1.6 Python implementation

`distance.py` (Appendix B) implements both numerical methods for any point and any curve:

- `analytical_parabola(x0, y0, a, b, c)` — for $y = ax^2 + bx + c$, condition (3) expands to the cubic

$$2a^2x^3 + 3abx^2 + (b^2 + 2a(c - y_0) + 1)x + (b(c - y_0) - x_0) = 0, \quad (26)$$

solved by a hand-written Cardano solver; the real root with the smallest D is returned.

- `newton_distance(...)` — equation (15). If `df/ddf` are omitted, central differences are used; `domain=(lo,hi)` clips iterates into the domain of f . Every step is recorded, which is what the step plots draw.
- `golden_section(...)` — the bracket method, recording every bracket.
- `distance_to_parabola(...)` and `distance_to_curve(...)` — run all methods on any point and any parabola, or on any f without derivatives.

```
>>> from distance import distance_to_parabola
>>> distance_to_parabola(-4, 0, a=1, b=0, c=5)
{'analytical': (6.289845364298704, -0.35546969287483443),
 'newton': (6.289845364298704, -0.3554696928748345, 4), # (d, x*, iterations)
 'golden': (6.2898453642987056, -0.3554696829624632, 40)}
```

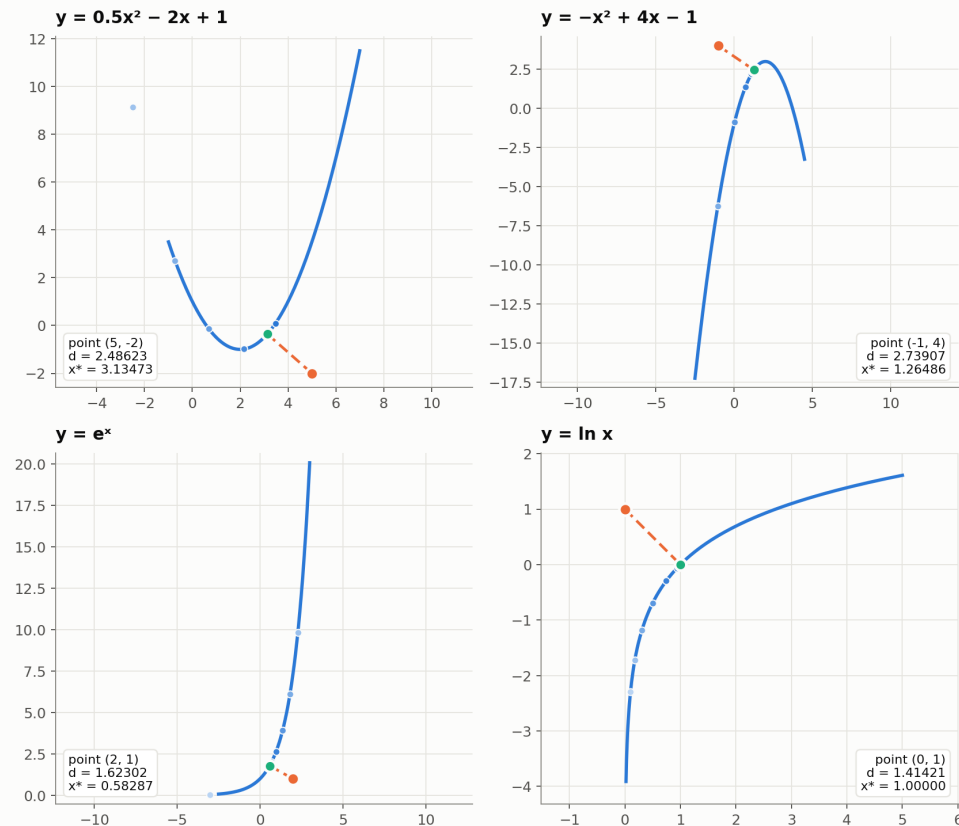
1.7 Other parabolas and non-polynomial functions

The same routines were applied to two further parabolas and to exponential, logarithmic, radical, rational (variable in the denominator) and cube-root functions. Each result is checked against a brute-force search over 400,001 grid points, and the parabolas also against (26).

Table 6: Other curves and points. All methods agree; d (grid) is the brute-force reference.

$y = f(x)$	type	(x_0, y_0)	x^*	d (Newton)	iters	d (golden)	d (grid)	d (exact)
$y = \frac{1}{2}x^2 - 2x + 1$	parabola	$(5, -2)$	3.134728	2.486228	10	2.486228	2.486228	2.486228
$y = -x^2 + 4x - 1$	parabola	$(-1, 4)$	1.264861	2.739072	9	2.739072	2.739072	2.739072
$y = e^x$	exponential	$(2, 1)$	0.582871	1.623025	9	1.623025	1.623025	—
$y = \ln x$	logarithmic	$(0, 1)$	1.000000	1.414214	9	1.414214	1.414214	—
$y = \sqrt{x}$	radical	$(4, 0)$	3.500000	1.936492	3	1.936492	1.936492	—
$y = \frac{1}{x}$	rational (x in denom.)	$(0, 0)$	1.000000	1.414214	11	1.414214	1.414214	—
$y = \frac{4}{1+x^2}$	rational (x in denom.)	$(3, 3)$	1.000000	2.236068	6	2.236068	2.236068	—
$y = \sqrt[3]{x}$	cube root	$(-1, 2)$	0.230350	1.854055	39	1.854055	1.854055	—

Three changes cover the non-polynomial cases. **Domain:** $\ln x$, $\sqrt{x}$ and $1/x$ exist only for $x > 0$, so Newton steps are clipped back into the domain and the bracket is chosen inside it. **Derivatives:** f' and f'' come from central differences ($h = 10^{-5}$, 10^{-4}), so no calculus is needed per function. **Smoothness:** $\sqrt[3]{x}$ has infinite slope at $x = 0$, so Newton needs 39 iterations instead of about 5, while golden section is unaffected. Note also that $D'(x) = 0$ is no longer solvable in closed form: for $y = e^x$ and $(2, 1)$ it reads $(x - 2) + (e^x - 1)e^x = 0$.

Same code, other parabolas, exponential, logarithm — Newton iterates (light → dark) and the shortest segment**Figure 5:** Two further parabolas, an exponential and a logarithm. Small dots are Newton iterates (light → dark); the dashed segment is the shortest distance.

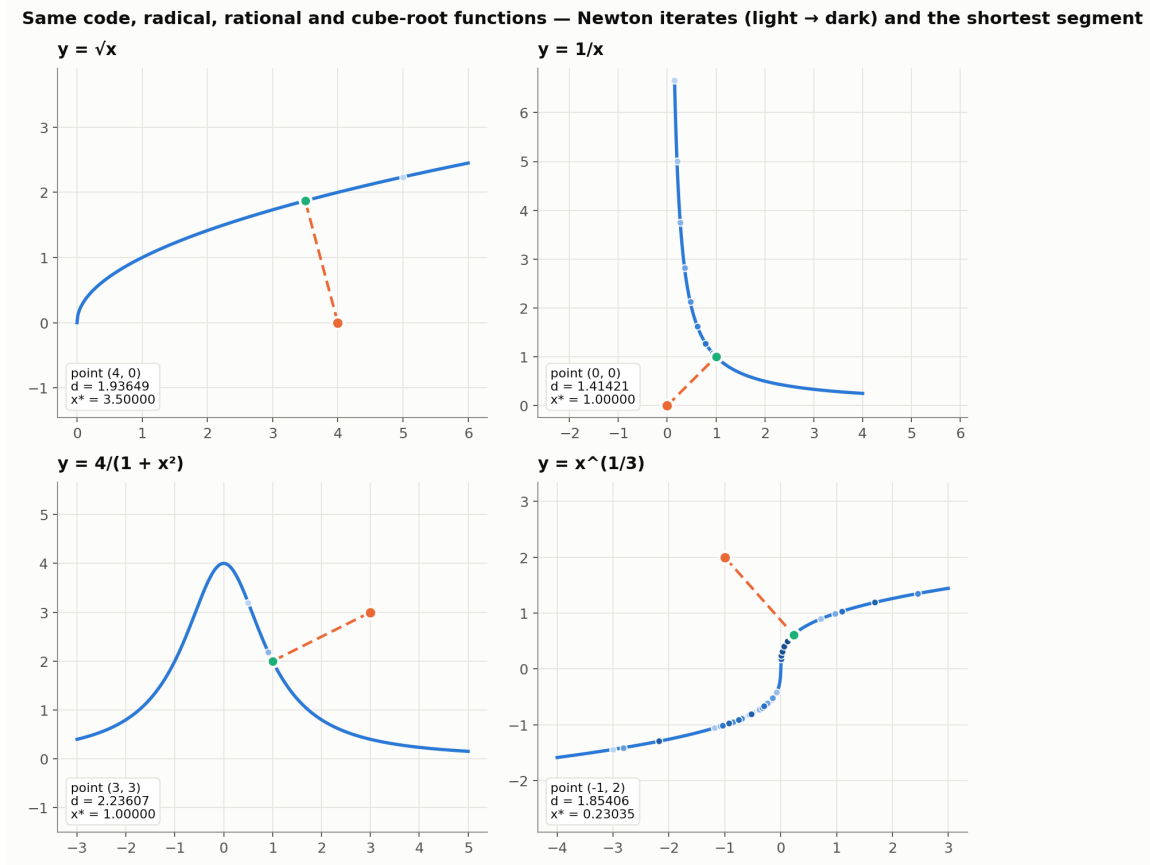


Figure 6: A radical, two rational functions (x in the denominator) and a cube root, handled by the same routines with a domain restriction where needed.

2 Fitting a line and a parabola

Problem. For $(0, 0.5)$, $(2, 3.5)$, $(1, 1.5)$, $(3, 7.5)$, fit a line and a parabola by minimising the mean squared error. Set up the MSE, solve analytically and with multivariate Newton–Raphson optimising one parameter at a time, and report the MSE of each fit. Supply code and plots of the intermediate steps.

2.1 Setting up the MSE

With $n = 4$ points,

$$\text{MSE}_{\text{line}}(m, b) = \frac{1}{n} \sum_{i=1}^n (mx_i + b - y_i)^2, \quad (27)$$

$$\text{MSE}_{\text{parab}}(a, b, c) = \frac{1}{n} \sum_{i=1}^n (ax_i^2 + bx_i + c - y_i)^2. \quad (28)$$

Both are quadratic in their parameters, so each has a unique minimiser. The required sums:

Table 7: Sums for the normal equations (the ordering of the points is irrelevant).

x_i	y_i	x_i^2	x_i^3	x_i^4	$x_i y_i$	$x_i^2 y_i$
0	0.5	0	0	0	0	0
1	1.5	1	1	1	1.5	1.5
2	3.5	4	8	16	7	14
3	7.5	9	27	81	22.5	67.5
$\Sigma = 6$	13	14	36	98	31	83

Differentiating (27) and substituting the sums:

$$\frac{\partial \text{MSE}}{\partial m} = \frac{2}{n} \sum_i (mx_i + b - y_i)x_i = \frac{1}{2}(14m + 6b - 31), \quad \frac{\partial^2 \text{MSE}}{\partial m^2} = 7, \quad (29)$$

$$\frac{\partial \text{MSE}}{\partial b} = \frac{2}{n} \sum_i (mx_i + b - y_i) = \frac{1}{2}(6m + 4b - 13), \quad \frac{\partial^2 \text{MSE}}{\partial b^2} = 2, \quad (30)$$

with cross term $\partial^2 \text{MSE} / \partial m \partial b = 3$. The parabola follows the same pattern with one more parameter.

2.2 Analytical solution

Line. Setting (29) and (30) to zero gives the normal equations

$$\begin{bmatrix} 14 & 6 \\ 6 & 4 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} = \begin{bmatrix} 31 \\ 13 \end{bmatrix}, \quad \det = 14(4) - 6(6) = 20, \quad (31)$$

$$m = \frac{4(31) - 6(13)}{20} = \frac{46}{20} = \mathbf{2.3}, \quad b = \frac{14(13) - 6(31)}{20} = \frac{-4}{20} = \mathbf{-0.2}. \quad (32)$$

Fitted values -0.2 , 2.1 , 4.4 , 6.7 at $x = 0, 1, 2, 3$; residuals -0.7 , 0.6 , 0.9 , -0.8 ; so

$$\text{MSE}_{\text{line}} = \frac{1}{4}(0.49 + 0.36 + 0.81 + 0.64) = \frac{2.30}{4} = \mathbf{0.575}. \quad (33)$$

Parabola. The three normal equations, reduced by Gaussian elimination:

$$\left[\begin{array}{ccc|c} 98 & 36 & 14 & 83 \\ 36 & 14 & 6 & 31 \\ 14 & 6 & 4 & 13 \end{array} \right] \longrightarrow \left[\begin{array}{ccc|c} 98 & 36 & 14 & 83 \\ 0 & \frac{38}{49} & \frac{6}{7} & \frac{25}{49} \\ 0 & 0 & \frac{20}{19} & \frac{11}{19} \end{array} \right] \quad (34)$$

Back-substitution:

$$c = \frac{11}{19} \cdot \frac{19}{20} = \frac{11}{20} = \mathbf{0.55}, \quad (35)$$

$$38b = 25 - 42(0.55) = 1.9 \Rightarrow b = \mathbf{0.05}, \quad (36)$$

$$98a = 83 - 36(0.05) - 14(0.55) = 73.5 \Rightarrow a = \mathbf{0.75}. \quad (37)$$

Fitted values 0.55, 1.35, 3.65, 7.45; residuals 0.05, -0.15, 0.15, -0.05; so

$$\text{MSE}_{\text{parab}} = \frac{1}{4}(0.0025 + 0.0225 + 0.0225 + 0.0025) = \frac{0.05}{4} = \mathbf{0.0125}. \quad (38)$$

Answers. Line: $y = 2.3x - 0.2$ with $\text{MSE} = 0.575$. Parabola: $y = 0.75x^2 + 0.05x + 0.55$ with $\text{MSE} = 0.0125$.

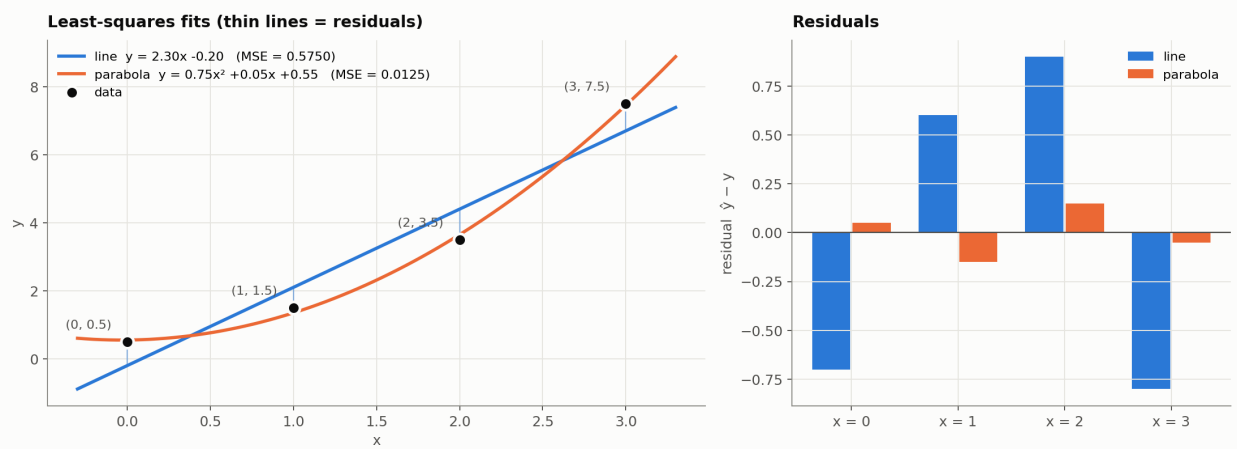


Figure 7: The two least-squares fits and their residuals. The line residuals alternate in sign, the signature of the curvature the parabola captures.

2.3 Numerical solution: Newton–Raphson, one parameter per step

Each step optimises one parameter with the others held fixed, by a one-dimensional Newton–Raphson step on that parameter’s partial derivative:

$$\theta_j \leftarrow \theta_j - \frac{\partial \text{MSE} / \partial \theta_j}{\partial^2 \text{MSE} / \partial \theta_j^2}, \quad j = 1, 2, \dots \text{ cyclically}. \quad (39)$$

For the line, substituting (29) and (30),

$$m \leftarrow m - \frac{\frac{1}{2}(14m + 6b - 31)}{7} = \frac{31 - 6b}{14}, \quad b \leftarrow b - \frac{\frac{1}{2}(6m + 4b - 13)}{2} = \frac{13 - 6m}{4}, \quad (40)$$

and for the parabola

$$a \leftarrow \frac{83 - 36b - 14c}{98}, \quad b \leftarrow \frac{31 - 36a - 6c}{14}, \quad c \leftarrow \frac{13 - 14a - 6b}{4}. \quad (41)$$

Since the MSE is quadratic in each parameter separately, each one-dimensional Newton step lands exactly on that parameter's minimiser given the others.

Line, starting from $m = b = 0$.

$$m_1 = \frac{31 - 6(0)}{14} = 2.214286, \quad b_1 = \frac{13 - 6(2.214286)}{4} = -0.071429, \quad (42)$$

$$m_2 = \frac{31.428571}{14} = 2.244898, \quad b_2 = \frac{13 - 6(2.244898)}{4} = -0.117347, \quad (43)$$

$$m_3 = \frac{31.704082}{14} = 2.264577, \quad b_3 = -0.146866. \quad (44)$$

Substituting the b update into the m update,

$$m_{k+1} = \frac{31 - 6\left(\frac{13 - 6m_k}{4}\right)}{14} = \frac{11.5 + 9m_k}{14}, \quad (45)$$

a linear fixed-point iteration with fixed point $m = 11.5/5 = 2.3$ and contraction ratio $9/14 = 0.642857$, so the error falls by about 36% per sweep. Convergence is linear, not quadratic, because the steps are restricted to the coordinate directions and ignore the cross term $\partial^2 \text{MSE} / \partial m \partial b = 3$.

Parabola, starting from $a = b = c = 0$.

$$a_1 = \frac{83}{98} = 0.846939, \quad b_1 = \frac{31 - 36(0.846939)}{14} = 0.036443, \quad (46)$$

$$c_1 = \frac{13 - 14(0.846939) - 6(0.036443)}{4} = 0.231050, \quad \text{MSE} = 0.117988, \quad (47)$$

$$a_2 = \frac{83 - 36(0.036443) - 14(0.231050)}{98} = 0.800544, \quad b_2 = 0.056723, \quad (48)$$

$$c_2 = \frac{13 - 14(0.800544) - 6(0.056723)}{4} = 0.363013, \quad \text{MSE} = 0.046400. \quad (49)$$

Table 8: Coordinate-wise Newton–Raphson for the line, from $m = b = 0$.

sweep	updated	m	b	MSE
0	start	0.000000	0.000000	17.750000
1	m	2.214286	0.000000	0.589286
1	b	2.214286	-0.071429	0.584184
2	m	2.244898	-0.071429	0.580904
2	b	2.244898	-0.117347	0.578795
3	m	2.264577	-0.117347	0.577440
3	b	2.264577	-0.146866	0.576568
4	m	2.277228	-0.146866	0.576008
4	b	2.277228	-0.165842	0.575648
5	m	2.285361	-0.165842	0.575417
5	b	2.285361	-0.178042	0.575268
6	m	2.290589	-0.178042	0.575172
6	b	2.290589	-0.185884	0.575111
⋮ converged after 48 sweeps to $m = 2.3000$, $b = -0.2000$, $\text{MSE} = 0.5750$ ⋮				

Table 9: Coordinate-wise Newton–Raphson for the parabola, from $a = b = c = 0$.

sweep	updated	a	b	c	MSE
0	start	0.000000	0.000000	0.000000	17.750000
1	a	0.846939	0.000000	0.000000	0.176020
1	b	0.846939	0.036443	0.000000	0.171372
1	c	0.846939	0.036443	0.231050	0.117988
2	a	0.800544	0.036443	0.231050	0.065253
2	b	0.800544	0.056722	0.231050	0.063814
2	c	0.800544	0.056722	0.363012	0.046400
3	a	0.774243	0.056722	0.363012	0.029452
3	b	0.774243	0.067798	0.363012	0.029023
3	c	0.774243	0.067798	0.438452	0.023332
4	a	0.759398	0.067798	0.438452	0.017932
4	b	0.759398	0.073641	0.438452	0.017813
4	c	0.759398	0.073641	0.481647	0.015947
$\vdots$ converged after 476 sweeps to $a = 0.7500$, $b = 0.0500$, $c = 0.5500$, $MSE = 0.0125$ $\vdots$					

Comparison: the full multivariate Newton step. Updating all parameters together, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - H^{-1}\nabla\text{MSE}$, uses the cross terms and minimises a quadratic in one step:

$$\begin{bmatrix} m \\ b \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 7 & 3 \\ 3 & 2 \end{bmatrix}^{-1} \begin{bmatrix} -15.5 \\ -6.5 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} 2 & -3 \\ -3 & 7 \end{bmatrix} \begin{bmatrix} 15.5 \\ 6.5 \end{bmatrix} = \begin{bmatrix} 2.3 \\ -0.2 \end{bmatrix}. \quad (50)$$

The parabola’s normal matrix is far worse conditioned (≈ 363 against ≈ 14 for the line), so its coordinate iteration needs about 480 sweeps to reach 10^{-10} against about 50 for the line, while the full step needs one for either model.

2.4 Results

Table 10: All methods reach the same optimum; they differ only in cost.

model	method	parameters	MSE	cost
Line	Analytical (normal equations)	$m = 2.3000$, $b = -0.2000$	0.5750	—
Line	Newton, one parameter at a time	$m = 2.3000$, $b = -0.2000$	0.5750	48 sweeps
Line	Newton, full multivariate step	$m = 2.3000$, $b = -0.2000$	0.5750	1 step
Parabola	Analytical (normal equations)	$a = 0.7500$, $b = 0.0500$, $c = 0.5500$	0.0125	—
Parabola	Newton, one parameter at a time	$a = 0.7500$, $b = 0.0500$, $c = 0.5500$	0.0125	476 sweeps
Parabola	Newton, full multivariate step	$a = 0.7500$, $b = 0.0500$, $c = 0.5500$	0.0125	1 step

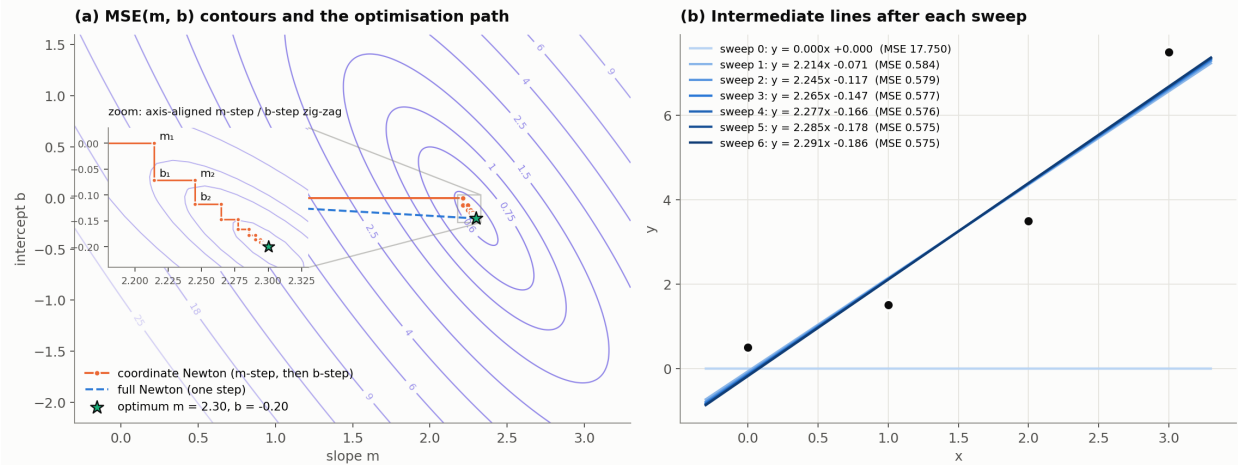


Figure 8: (a) Contours of $MSE(m, b)$ with the optimisation paths: the coordinate scheme moves parallel to the m axis, then the b axis, each move ending tangent to a contour (inset), while the full Newton step goes straight to the optimum. (b) The fitted line after each sweep.

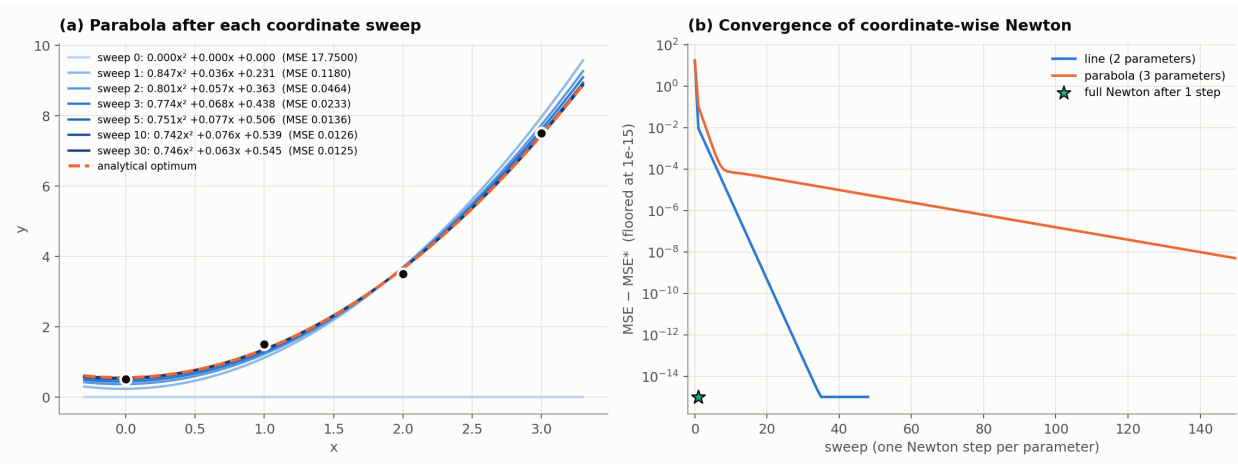


Figure 9: (a) The parabola after sweeps 0–30; most of the improvement happens in the first three. (b) Excess MSE per sweep on a log scale, with the full Newton step marked.

2.5 Python implementation

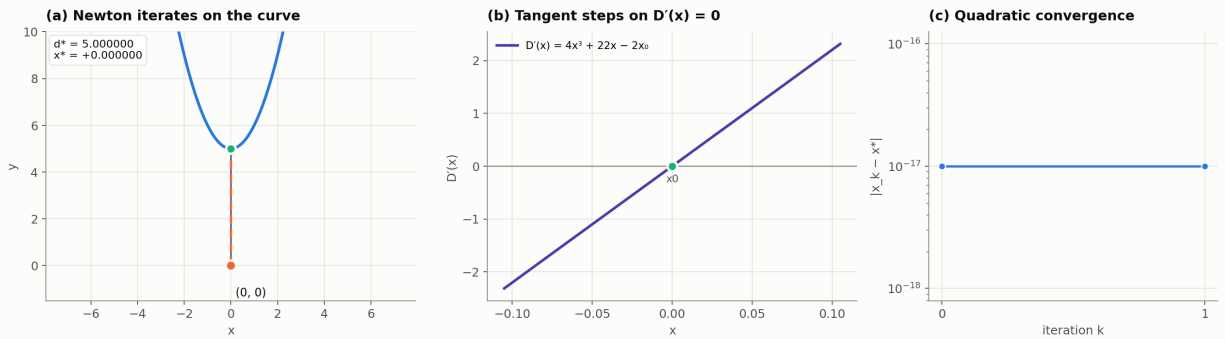
`fitting.py` (Appendix C) writes both models as $\hat{y} = \Phi\theta$, with $\Phi = [\mathbf{x} \ 1]$ for the line and $[\mathbf{x}^2 \ \mathbf{x} \ 1]$ for the parabola, so one gradient and one Hessian serve both models and any degree:

$$\nabla \text{MSE} = \frac{2}{n} \Phi^T (\Phi\theta - \mathbf{y}), \quad H = \frac{2}{n} \Phi^T \Phi. \quad (51)$$

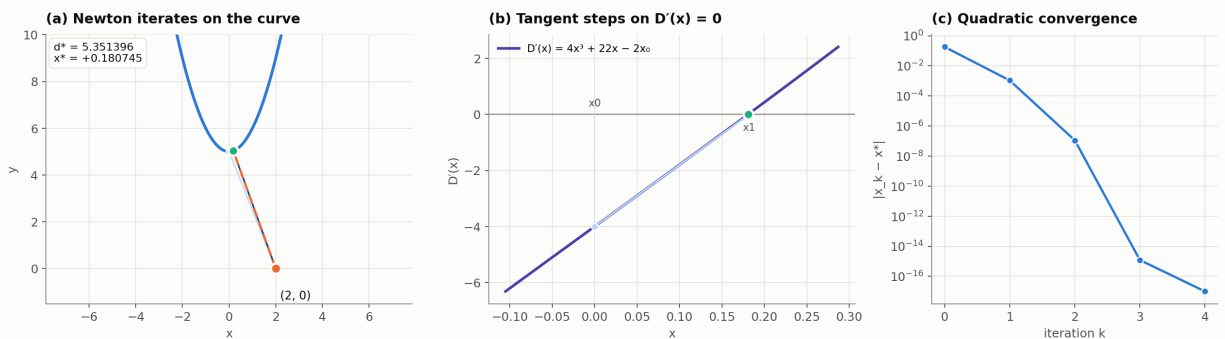
The three solvers are `normal_equations` (analytical, via a hand-written Gaussian elimination), `coordinate_newton` (equation (39), which records every sub-step), and `full_newton` for comparison.

A Intermediate-step plots for the remaining points

Newton-Raphson · point (0, 0) · initial guess $x = 0$



Newton-Raphson · point (2, 0) · initial guess $x = 0$



Newton-Raphson · point (6, 0) · initial guess $x = 0$

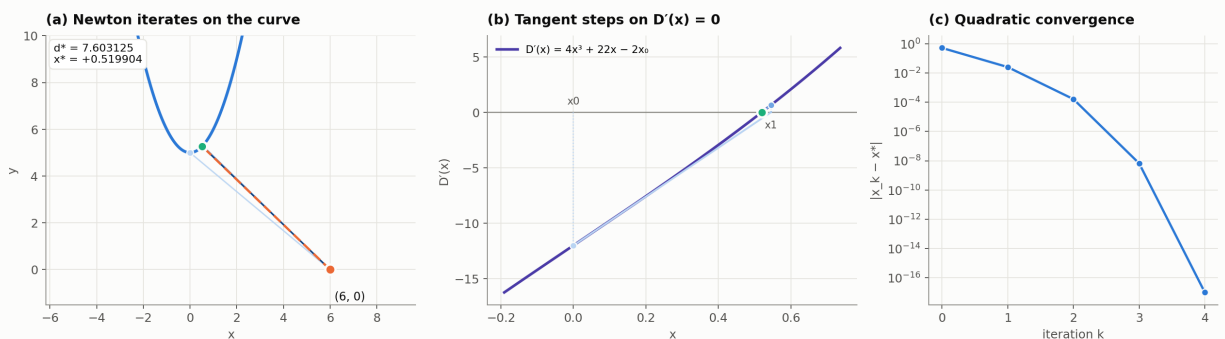
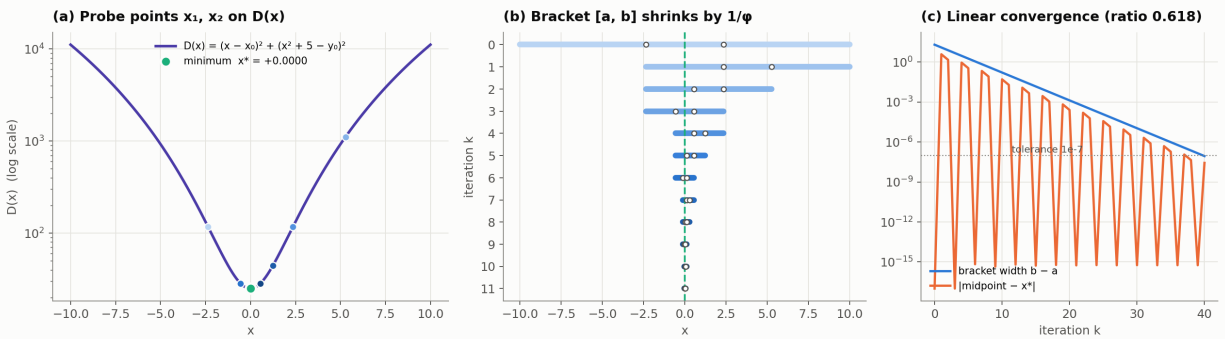


Figure 10: Newton-Raphson for (0,0), (2,0) and (6,0). For (0,0) the initial guess is already the solution.

Golden Section Search · point (0, 0) · bracket [-10, 10]



Golden Section Search · point (-8, 0) · bracket [-10, 10]

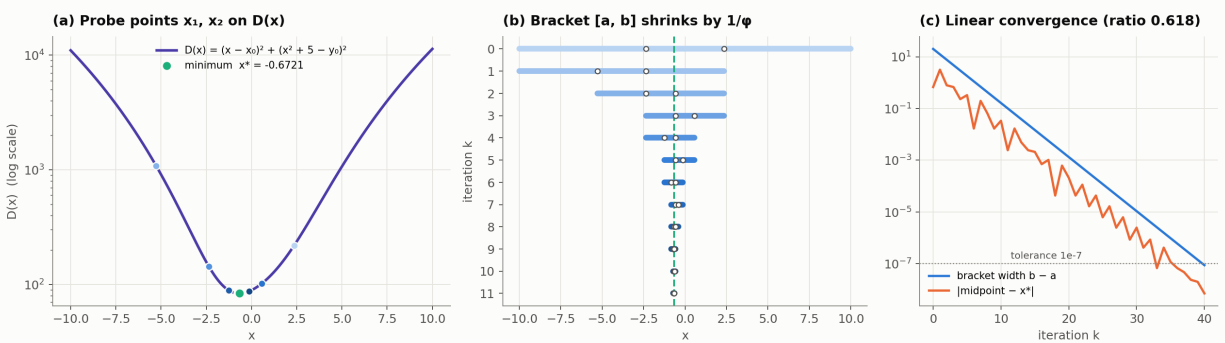


Figure 11: Golden section search for (0, 0) and (-8, 0).

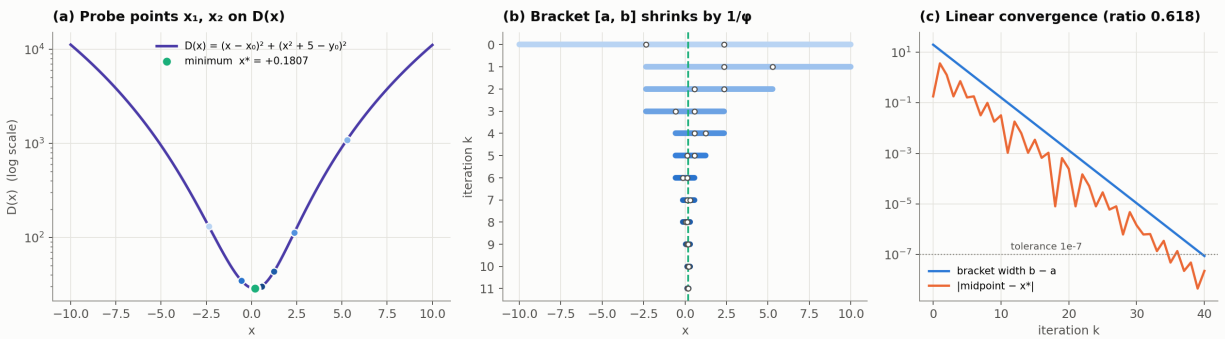
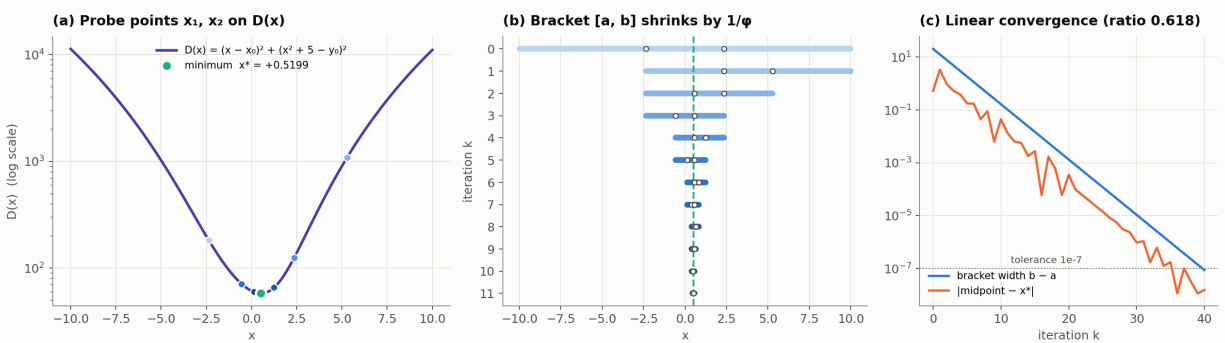
Golden Section Search · point (2, 0) · bracket $[-10, 10]$ Golden Section Search · point (6, 0) · bracket $[-10, 10]$ 

Figure 12: Golden section search for (2, 0) and (6, 0).

B distance.py — Part 1 code

```

1  """
2  SYDE 572 - Assignment 1, Part 1
3  =====
4  Shortest distance from a point (x0, y0) to a curve y = f(x).
5
6  Run this file to reproduce every Part 1 result:
7
8      python distance.py
9
10 It prints the result tables and writes all Part 1 figures to ../media/.
11
12 Contents
13 -----
14 Section 1 The objective function D(x) and its derivatives
15 Section 2 Analytical solution for a parabola (Cardano's cubic formula)
16 Section 3 Numerical method A: Newton-Raphson on D'(x) = 0
17 Section 4 Numerical method B: Golden section search on D(x)
18 Section 5 Convenience wrappers (any point, any parabola, any function)
19 Section 6 Plotting helpers
20 Section 7 Demonstrations: the assigned points, other curves, non-polynomial
21                functions, and a case with several stationary points
22
23 Only numpy and matplotlib are used, and no optimisation library: the three
24 solvers, the cubic solver and the finite-difference derivatives are all written
25 out here.
26 """
27
28 import math
29 import os
30
31 import matplotlib
32 matplotlib.use("Agg") # write PNG files, no interactive window
33 import matplotlib.pyplot as plt
34 import numpy as np
35 from matplotlib.colors import LinearSegmentedColormap
36
37 # The golden ratio and the fraction of the bracket used by golden section search.
38 PHI = (1 + math.sqrt(5)) / 2 # 1.618034
39 RESPHI = 2 - PHI # 1/phi^2 = 0.381966
40
41 MEDIA = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "media")
42
43 # =====
44 # Section 1. The objective function
45 # =====
46
47 def dist_sq(x, x0, y0, f):
48     """Squared distance D(x) from the point (x0, y0) to the curve point (x, f(x)).
49
50     The true distance is sqrt(D(x)). Because the square root is monotonically
51     increasing, the x that minimises D also minimises the distance, so every
52     solver below works with D and takes the square root only at the very end.
53     """
54     return (x - x0) ** 2 + (f(x) - y0) ** 2
55
56
57 def _finite(v):
58     """True if v is a usable number (not NaN or +-inf)."""
59     return isinstance(v, (int, float, np.floating)) and math.isfinite(float(v))
60
61
62 def numeric_d1(f, h=1e-5):
63     """First derivative of f by central differences: used when f' is unknown.
64

```

```

65     Returned as a function so it can be dropped in wherever an analytic
66     derivative would go.  $h = 1e-5$  keeps the truncation error (order  $h^2$ ) and
67     the floating-point cancellation error (order  $\text{eps}/h$ ) comparably small.
68
69     Near the edge of the domain (e.g.  $\ln x$  at  $x = 1e-6$ ) one of the two sample
70     points can fall outside it and return NaN, so the routine falls back to a
71     one-sided difference there.
72     """
73     def d1(x):
74         fp, fm_ = f(x + h), f(x - h)
75         if _finite(fp) and _finite(fm_):
76             return (fp - fm_) / (2 * h)
77         if _finite(fp): # forward difference
78             return (fp - f(x)) / h
79         return (f(x) - fm_) / h # backward difference
80     return d1
81
82
83     def numeric_d2(f, h=1e-4):
84         """Second derivative of f by central differences (larger h: divides by  $h^2$ ).
85
86         Same domain-edge fallback as numeric_d1: if a sample point is outside the
87         domain, the stencil is shifted to one side.
88         """
89         def d2(x):
90             f0, fp, fm_ = f(x), f(x + h), f(x - h)
91             if _finite(fp) and _finite(fm_):
92                 return (fp - 2 * f0 + fm_) / (h * h)
93             if _finite(fp): # forward stencil
94                 return (f(x + 2 * h) - 2 * fp + f0) / (h * h)
95             return (f0 - 2 * fm_ + f(x - 2 * h)) / (h * h)
96         return d2
97
98
99     # =====
100     # Section 2. Analytical solution for a parabola
101     # =====
102     def _cbrrt(v):
103         """Real cube root, including for negative v ( $v^{1/3}$  fails for  $v < 0$ )."""
104         return math.copysign(abs(v) ** (1 / 3), v)
105
106
107     def solve_cubic(A, B, C, D):
108         """All real roots of  $Ax^3 + Bx^2 + Cx + D = 0$  ( $A \neq 0$ ).
109
110         Substituting  $x = t - B/(3A)$  removes the quadratic term and leaves the
111         depressed cubic  $t^3 + pt + q = 0$ , which Cardano's formula solves. The sign
112         of the discriminant decides how many real roots there are:
113
114          $\text{disc} > 0 \rightarrow$  one real root (two cube roots added)
115          $\text{disc} < 0 \rightarrow$  three real roots (trigonometric form; cube roots of a
116             complex number would be needed instead)
117          $\text{disc} = 0 \rightarrow$  a repeated root
118         """
119         b, c, d = B / A, C / A, D / A
120         p = c - b * b / 3
121         q = 2 * b ** 3 / 27 - b * c / 3 + d
122         shift = -b / 3 # undo the substitution later
123         disc = (q / 2) ** 2 + (p / 3) ** 3
124
125         if disc > 1e-14: # one real root
126             s = math.sqrt(disc)
127             t = [_cbrrt(-q / 2 + s) + _cbrrt(-q / 2 - s)]
128         elif disc < -1e-14: # three distinct real roots
129             r = 2 * math.sqrt(-p / 3)
130             phi = math.acos(3 * q / (p * r))
131             t = [r * math.cos((phi - 2 * math.pi * k) / 3) for k in range(3)]
132         else: # repeated roots

```

```

133     u = _cbqrt(-q / 2)
134     t = [2 * u, -u]
135
136     return sorted(ti + shift for ti in t)
137
138
139 def analytical_parabola(x0, y0, a, b, c):
140     """Exact closest point on the parabola  $y = a x^2 + b x + c$  to  $(x0, y0)$ .
141
142     Setting  $D'(x) = 0$  gives
143          $(x - x0) + (a x^2 + b x + c - y0)(2a x + b) = 0$ ,
144     which expands to the cubic
145          $2a^2 x^3 + 3ab x^2 + (b^2 + 2a(c - y0) + 1) x + (b(c - y0) - x0) = 0$ .
146
147     Every real root is a stationary point of  $D$ , so the root with the smallest  $D$ 
148     is the answer ( $D \rightarrow \infty$  as  $|x| \rightarrow \infty$ , so a minimum always exists).
149
150     If  $a = 0$  the "parabola" is the straight line  $y = b x + c$ , for which the
151     condition is linear and gives  $x^* = (x0 + b(y0 - c)) / (1 + b^2)$  directly.
152
153     Returns (distance,  $x^*$ , all real roots).
154     """
155     f = lambda x: a * x * x + b * x + c
156     k = c - y0
157
158     if a == 0: # degenerate case: a line
159         xs = (x0 + b * (y0 - c)) / (1 + b * b)
160         return math.sqrt(dist_sq(xs, x0, y0, f)), xs, [xs]
161
162     roots = solve_cubic(2 * a * a, 3 * a * b, b * b + 2 * a * k + 1, b * k - x0)
163     best = min(roots, key=lambda x: dist_sq(x, x0, y0, f))
164     return math.sqrt(dist_sq(best, x0, y0, f)), best, roots
165
166
167 # =====
168 # Section 3. Numerical method A: Newton-Raphson
169 # =====
170 def newton_distance(x0, y0, f, df=None, ddf=None, initial_guess=0.0,
171                    tolerance=1e-7, max_iter=100, domain=None):
172     """Newton-Raphson applied to the root-finding problem  $D'(x) = 0$ .
173
174          $D'(x) = 2(x - x0) + 2(f(x) - y0) f'(x)$ 
175          $D''(x) = 2 + 2 f'(x)^2 + 2(f(x) - y0) f''(x)$ 
176          $x_{k+1} = x_k - D'(x_k) / D''(x_k)$ 
177
178     df, ddf : derivatives of f. If omitted, central differences are used, so
179     the routine works for any f the caller can evaluate.
180     domain : (lo, hi) to clip each iterate into the domain of f - needed for
181     ln x, sqrt(x), 1/x, where an unconstrained step can overshoot
182     into  $x \leq 0$  and produce NaN.
183     tolerance : stop once the step is smaller than this.
184
185     Returns (distance,  $x^*$ , history), where history holds  $x_k$ ,  $D$ ,  $D'$ ,  $D''$  and the
186     next iterate for every step, which is what the step plots are drawn from.
187     """
188     df = df or numeric_d1(f)
189     ddf = ddf or numeric_d2(f)
190
191     x = initial_guess
192     history = []
193     for k in range(max_iter):
194         d1 = 2 * (x - x0) + 2 * (f(x) - y0) * df(x)
195         d2 = 2 + 2 * df(x) ** 2 + 2 * (f(x) - y0) * ddf(x)
196
197         next_x = x - d1 / d2 # the Newton step itself
198         if domain is not None:
199             next_x = min(max(next_x, domain[0]), domain[1])
200

```

```

201     history.append(dict(k=k, x=x, D=dist_sq(x, x0, y0, f),
202                        d1=d1, d2=d2, next_x=next_x))
203     if abs(next_x - x) < tolerance:
204         x = next_x
205         break
206     x = next_x
207
208     return math.sqrt(dist_sq(x, x0, y0, f)), x, history
209
210
211 def newton_multistart(x0, y0, f, df=None, ddf=None, starts=(-5, -2, 0, 2, 5), **kw):
212     """Run Newton from several starting guesses and keep the closest result.
213
214     Newton converges to a stationary point of  $D$ , which may be a local maximum
215     or a non-global minimum (see multiple_minima_demo). Trying several starts
216     is the cheapest guard against that.
217     """
218     runs = [newton_distance(x0, y0, f, df, ddf, initial_guess=s, **kw) for s in starts]
219     # A run can diverge to NaN (e.g. a starting guess right on the edge of the
220     # domain of  $f$ ); those are discarded rather than compared, since any
221     # comparison against NaN is False and would corrupt the minimum.
222     usable = [r for r in runs if math.isfinite(r[0])]
223     if not usable:
224         raise ValueError("every Newton start diverged; check the starting guesses or the domain")
225     return min(usable, key=lambda r: r[0])
226
227
228 # =====
229 # Section 4. Numerical method B: Golden section search
230 # =====
231 def golden_section(x0, y0, f, a, b, tolerance=1e-7, max_iter=500):
232     """Golden section search for the minimum of  $D$  on the bracket  $[a, b]$ .
233
234     Two interior probes are placed symmetrically:
235          $x_1 = a + r(b - a)$ ,  $x_2 = b - r(b - a)$ ,  $r = 2 - \phi = 0.381966$ .
236     If  $D(x_1) < D(x_2)$  the minimum of a unimodal  $D$  cannot lie in  $(x_2, b]$ , so the
237     bracket becomes  $[a, x_2]$ ; otherwise it becomes  $[x_1, b]$ . The golden ratio is
238     chosen so the surviving probe is already in the right place for the new
239     bracket, so each iteration costs one new evaluation of  $D$  and shrinks the
240     bracket by  $1/\phi = 0.618$ .
241
242     No derivatives are used anywhere, which is the whole point of this method.
243
244     Returns (distance,  $x^*$ , history) with the bracket and both probes per step.
245     """
246     D = lambda x: dist_sq(x, x0, y0, f)
247
248     x1 = a + RESPHI * (b - a)
249     x2 = b - RESPHI * (b - a)
250     f1, f2 = D(x1), D(x2)
251
252     history = []
253     k = 0
254     while abs(b - a) > tolerance and k < max_iter:
255         history.append(dict(k=k, a=a, b=b, x1=x1, x2=x2, D1=f1, D2=f2))
256         if f1 < f2: # minimum is in  $[a, x_2]$ ;  $x_1$  becomes the new  $x_2$ 
257             b, x2, f2 = x2, x1, f1
258             x1 = a + RESPHI * (b - a)
259             f1 = D(x1)
260         else: # minimum is in  $[x_1, b]$ ;  $x_2$  becomes the new  $x_1$ 
261             a, x1, f1 = x1, x2, f2
262             x2 = b - RESPHI * (b - a)
263             f2 = D(x2)
264         k += 1
265
266     history.append(dict(k=k, a=a, b=b, x1=x1, x2=x2, D1=f1, D2=f2))
267     best_x = (a + b) / 2 # the midpoint of the final bracket
268     return math.sqrt(D(best_x)), best_x, history

```

```

269
270
271 # =====
272 # Section 5. Convenience wrappers
273 # =====
274 def parabola(a, b, c):
275     """Return f, f', f'' for y = a x^2 + b x + c."""
276     return (lambda x: a * x * x + b * x + c,
277             lambda x: 2 * a * x + b,
278             lambda x: 2 * a)
279
280
281 def distance_to_parabola(x0, y0, a, b, c, bracket=(-10, 10)):
282     """Distance from any point to any parabola, by all three methods.
283
284     >>> r = distance_to_parabola(-4, 0, a=1, b=0, c=5)
285     >>> round(r['analytical'][0], 6), round(r['newton'][0], 6), round(r['golden'][0], 6)
286     (6.289845, 6.289845, 6.289845)
287     """
288     f, df, ddf = parabola(a, b, c)
289     da, xa, _ = analytical_parabola(x0, y0, a, b, c)
290     dn, xn, hn = newton_distance(x0, y0, f, df, ddf, initial_guess=0.0)
291     dg, xg, hg = golden_section(x0, y0, f, *bracket)
292     return dict(analytical=(da, xa),
293                newton=(dn, xn, len(hn)),
294                golden=(dg, xg, len(hg) - 1))
295
296
297 def distance_to_curve(x0, y0, f, bracket, domain=None, starts=None):
298     """Distance from any point to any curve y = f(x), without needing f' or f''.
299
300     This is the "slight modification" asked for in the assignment: the
301     derivatives are estimated numerically and the search is confined to the
302     domain of f, so exponential, logarithmic, rational and radical functions all
303     work with the same two solvers.
304     """
305     lo, hi = bracket
306     if starts is None:
307         starts = sorted({min(max(x0, lo + 0.1), hi - 0.1), (lo + hi) / 2,
308                          lo + 0.25 * (hi - lo), lo + 0.75 * (hi - lo)})
309     dn, xn, hn = newton_multistart(x0, y0, f, starts=starts, domain=domain)
310     dg, xg, hg = golden_section(x0, y0, f, lo, hi)
311
312     # Brute-force reference: the minimum over a fine grid, used only as a check.
313     grid = np.linspace(lo, hi, 400001)
314     ref_x = float(grid[np.argmin(dist_sq(grid, x0, y0, f))])
315
316     return dict(newton=(dn, xn, len(hn)), newton_history=hn,
317                golden=(dg, xg, len(hg) - 1),
318                grid=(math.sqrt(dist_sq(ref_x, x0, y0, f)), ref_x))
319
320
321 # =====
322 # Section 6. Plotting helpers
323 # =====
324 INK, INK2, MUTED, GRID = "#0b0b0b", "#52514e", "#8a8984", "#e6e5e0"
325 BLUE, ORANGE, AQUA, VIOLET, RED = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
326 SURFACE = "#fcfcfb"
327 # Light -> dark blue ramp: early iterates are pale, the converged one is dark.
328 ITER_CMAP = LinearSegmentedColormap.from_list("iter", ["#b9d4f3", "#2a78d6", "#0f3c75"])
329
330 plt.rcParams.update({
331     "figure.dpi": 110, "savefig.dpi": 170,
332     "figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "savefig.facecolor": SURFACE,
333     "font.family": "DejaVu Sans", "font.size": 10.5,
334     "axes.edgecolor": MUTED, "axes.labelcolor": INK2, "axes.titlecolor": INK,
335     "axes.titlesize": 12, "axes.titleweight": "bold", "axes.titlelocation": "left",
336     "axes.titlepad": 10,

```

```

337 "axes.spines.top": False, "axes.spines.right": False,
338 "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.8,
339 "xtick.color": INK2, "ytick.color": INK2, "legend.frameon": False,
340 "lines.linewidth": 2, "mathtext.fontset": "dejavusans",
341 })
342
343
344 def save(fig, name):
345     """Save a figure into ../media/ and report it."""
346     os.makedirs(MEDIA, exist_ok=True)
347     fig.savefig(os.path.join(MEDIA, name), bbox_inches="tight", pad_inches=0.15)
348     plt.close(fig)
349     print(" figure:", name)
350
351
352 def draw_shortest_segment(ax, x0, y0, xs, f):
353     """Dashed segment from the query point to the closest curve point."""
354     ax.plot([x0, xs], [y0, f(xs)], color=ORANGE, lw=2, ls=(0, (4, 2)), zorder=4)
355     ax.scatter([xs], [f(xs)], s=70, color=AQUA, edgecolor=SURFACE, linewidth=2, zorder=6)
356
357
358 def draw_query_point(ax, x0, y0, text=None, dx=0.3, dy=-0.9):
359     ax.scatter([x0], [y0], s=80, color=ORANGE, edgecolor=SURFACE, linewidth=2, zorder=6)
360     if text:
361         ax.annotate(text, (x0, y0), xytext=(x0 + dx, y0 + dy), color=INK, fontsize=10)
362
363
364 def plot_overview(points, f, results, name="p1_overview.png"):
365     """All five query points, their closest curve points and the distances."""
366     fig, ax = plt.subplots(figsize=(9, 5.6))
367     xs = np.linspace(-3.6, 3.6, 400)
368     ax.plot(xs, f(xs), color=BLUE, lw=2.5, label=r"$y = x^2 + 5$", zorder=3)
369
370     for (x0, y0), r in zip(points, results):
371         xa = r["analytical"][1]
372         ax.plot([x0, xa], [y0, f(xa)], color=ORANGE, lw=1.8, ls=(0, (4, 2)), zorder=4)
373         ax.scatter([xa], [f(xa)], s=60, color=AQUA, edgecolor=SURFACE, linewidth=2, zorder=6)
374         ax.scatter([x0], [y0], s=80, color=ORANGE, edgecolor=SURFACE, linewidth=2, zorder=6)
375         ax.annotate(f"({x0}, {y0})\nd = {r['analytical'][0]:.4f}", (x0, y0),
376                     xytext=(0, -62 if x0 == 2 else -34), textcoords="offset points",
377                     ha="center", fontsize=9.5, color=INK)
378
379     # legend proxies
380     ax.plot([], [], color=ORANGE, lw=1.8, ls=(0, (4, 2)), label="shortest segment")
381     ax.scatter([], [], s=60, color=AQUA, label="closest point on curve")
382     ax.scatter([], [], s=80, color=ORANGE, label="query point")
383
384     ax.set_xlim(-9.5, 7.5); ax.set_ylim(-4.6, 13)
385     ax.set_aspect("equal") # so the right angles look like right angles
386     ax.set_xlabel("x"); ax.set_ylabel("y")
387     ax.set_title("Shortest distance from five points to  $y = x^2 + 5$ ")
388     ax.legend(loc="upper left", fontsize=9.5)
389     save(fig, name)
390
391
392 def plot_newton_steps(x0, y0, f, df, ddf, history, x_exact, x_final, d_final):
393     """Three panels of Newton's intermediate steps for one query point.
394
395     (a) the iterates projected onto the curve
396     (b) the tangent construction on  $D'(x)$ : each tangent is followed to its zero
397     (c)  $|x_k - x^*|$  on a log scale, showing the quadratic collapse
398     """
399     iterates = [h["x"] for h in history] + [x_final]
400     n = len(iterates)
401     cols = [ITER_CMAP(i / max(n - 1, 1)) for i in range(n)]
402
403     fig, axs = plt.subplots(1, 3, figsize=(15, 4.6),
404                             gridspec_kw=dict(width_ratios=[1.15, 1.15, 0.9]))

```

```

405
406 # ---- (a) geometry -----
407 ax = axs[0]
408 lo, hi = min(x0, -2.5) - 1, max(x0, 2.5) + 1
409 xx = np.linspace(lo, hi, 400)
410 ax.plot(xx, f(xx), color=BLUE, lw=2.5, label="y = x2 + 5")
411 for i, xi in enumerate(iterates):
412     ax.plot([x0, xi], [y0, f(xi)], color=cols[i], lw=1.2, alpha=0.9)
413     ax.scatter([xi], [f(xi)], s=36, color=cols[i], edgecolor=SURFACE, linewidth=1, zorder=5)
414 draw_shortest_segment(ax, x0, y0, x_exact, f)
415 draw_query_point(ax, x0, y0, f("{x0}, {y0}", dx=0.2, dy=-1.3)
416 ax.set_xlim(lo, hi); ax.set_ylim(-1.5, 10)
417 ax.set_aspect("equal", adjustable="datalim")
418 ax.set_xlabel("x"); ax.set_ylabel("y")
419 ax.set_title("(a) Newton iterates on the curve")
420 ax.text(0.02, 0.97, f"d* = {d_final:.6f} \n x* = {x_final:+.6f}", transform=ax.transAxes,
421        va="top", fontsize=9.5, color=INK,
422        bbox=dict(boxstyle="round,pad=0.4", fc="white", ec=GRID))
423
424 # ---- (b) the tangent steps on D'(x) -----
425 ax = axs[1]
426 Dp = lambda x: 2 * (x - x0) + 2 * (f(x) - y0) * df(x)
427 xl, xh = min(iterates), max(iterates)
428 pad = max(xh - xl, 0.3) * 0.35
429 xx = np.linspace(xl - pad, xh + pad, 400)
430 ax.axhline(0, color=MUTED, lw=1)
431 ax.plot(xx, Dp(xx), color=VIOLET, lw=2.3, label="D(x) = 4x3 + 22x - 2x")
432 for i, h in enumerate(history):
433     ax.plot([h["x"], h["x"]], [0, h["d1"]], color=cols[i], lw=1, ls=":")
434     ax.plot([h["x"], h["next_x"]], [h["d1"], 0], color=cols[i], lw=1.6)
435     ax.scatter([h["x"]], [h["d1"]], s=34, color=cols[i], zorder=5, edgecolor=SURFACE)
436     if i < 2:
437         ax.annotate(f"x{i}", (h["x"], 0), xytext=(0, 7 if h["d1"] < 0 else -14),
438                    textcoords="offset points", ha="center", fontsize=9, color=INK2)
439 ax.scatter([x_exact], [0], s=70, color=AQUA, edgecolor=SURFACE, linewidth=2, zorder=6)
440 ax.set_xlabel("x"); ax.set_ylabel("D(x)")
441 ax.set_title("(b) Tangent steps on D(x) = 0")
442 ax.legend(loc="upper left", fontsize=9)
443
444 # ---- (c) convergence -----
445 ax = axs[2]
446 err = [max(abs(xi - x_exact), 1e-17) for xi in iterates] # clip for the log axis
447 ax.semilogy(range(n), err, color=BLUE, marker="o", ms=6, mec=SURFACE)
448 ax.set_xlabel("iteration k"); ax.set_ylabel("|xk - x*|")
449 ax.set_title("(c) Quadratic convergence")
450 ax.set_xticks(range(n))
451
452 fig.suptitle(f"-NewtonRaphson · point ({x0}, {y0}) · initial guess x = 0",
453            x=0.01, ha="left", fontsize=13.5, fontweight="bold", color=INK, y=1.03)
454 fig.tight_layout()
455 save(fig, f"p1_newton_{x0}_{y0}.png")
456
457
458 def plot_golden_steps(x0, y0, f, history, x_exact, bracket):
459     """Three panels of the golden section intermediate steps for one point.
460
461     (a) the probe pairs on D(x)
462     (b) the bracket ladder: [a, b] for the first iterations
463     (c) bracket width and midpoint error, both decaying by 0.618 per step
464     """
465     D = lambda x: dist_sq(x, x0, y0, f)
466     nshow = 12
467     cols = [ITER_CMAP(i / (nshow - 1)) for i in range(nshow)]
468
469     fig, axs = plt.subplots(1, 3, figsize=(15, 4.6),
470                            gridspec_kw=dict(width_ratios=[1.2, 1.1, 0.9]))
471
472     # ---- (a) D(x) with the first probe pairs -----

```



```

541     axs[0].set_title("-NewtonRaphson: error per iteration")
542     axs[1].set_title("Golden section: error per iteration")
543     for ax in axs:
544         ax.set_xlabel("iteration k")
545         axs[0].set_ylabel("|x_k - x*|")
546         axs[0].legend(title="point", fontsize=9, title_fontsize=9)
547         axs[1].text(0.98, 0.97, "(0, 0) omitted: its midpoint equals x* = 0\n"
548                     "exactly on every other step (error = 0)",
549                     transform=axs[1].transAxes, ha="right", va="top", fontsize=8.5, color=INK2)
550     fig.tight_layout()
551     save(fig, name)
552
553
554     # =====
555     # Section 7. Demonstrations
556     # =====
557     ASSIGNED_POINTS = [(0, 0), (-4, 0), (-8, 0), (2, 0), (6, 0)]
558     BRACKET = (-10.0, 10.0) # starting bracket for golden section search
559
560
561     def assigned_points_demo():
562         """Part 1 as assigned: the five points and the parabola  $y = x^2 + 5$ .
563
564         Runs all three methods, prints the comparison table and draws the overview,
565         the per-point intermediate-step figures and the convergence comparison.
566         """
567         a, b, c = 1.0, 0.0, 5.0
568         f, df, ddf = parabola(a, b, c)
569
570         print("\n" + "=" * 78)
571         print("PART 1 ---  $y = x^2 + 5$ ")
572         print("=" * 78)
573         print(f"{'point':>9} | {'x* (exact)':>12} {'d (exact)':>11} | "
574               f"{'d (Newton)':>11} {'it':>3} | {'d (golden)':>11} {'it':>3}")
575         print("-" * 78)
576
577         results = []
578         for (x0, y0) in ASSIGNED_POINTS:
579             da, xa, roots = analytical_parabola(x0, y0, a, b, c)
580             dn, xn, hn = newton_distance(x0, y0, f, df, ddf, initial_guess=0.0)
581             dg, xg, hg = golden_section(x0, y0, f, *BRACKET)
582             results.append(dict(point=(x0, y0), analytical=(da, xa), roots=roots,
583                                newton=(dn, xn), newton_history=hn,
584                                golden=(dg, xg), golden_history=hg))
585             print(f"{'str((x0, y0))':>9} | {xa:12.6f} {da:11.6f} | "
586                   f"{dn:11.6f} {len(hn):3d} | {dg:11.6f} {len(hg) - 1:3d}")
587
588         # Per-point iterate tables, i.e. the intermediate steps in numbers.
589         print("\nNewton iterates (initial guess x = 0):")
590         for r in results:
591             xs = [h["x"] for h in r["newton_history"]] + [r["newton"][1]]
592             print(f"{'str(r['point']):>9}: " + " ".join(f"{v:+.7f}" for v in xs))
593
594         print("\nGolden section, first four brackets:")
595         for r in results:
596             print(f"{'str(r['point']):>9}: " +
597                   " ".join(f"{h['a']:+.4f}, {h['b']:+.4f}" for h in r["golden_history"][:4]))
598
599         # Figures.
600         plot_overview(ASSIGNED_POINTS, f, results)
601         for r in results:
602             x0, y0 = r["point"]
603             plot_newton_steps(x0, y0, f, df, ddf, r["newton_history"],
604                              r["analytical"][1], r["newton"][1], r["newton"][0])
605             plot_golden_steps(x0, y0, f, r["golden_history"], r["analytical"][1], BRACKET)
606             plot_convergence_comparison(ASSIGNED_POINTS, results)
607         return results
608

```

```

609
610 # Other curves: two more parabolas plus the non-polynomial cases required by the
611 # assignment (exponential, logarithmic, variable in the denominator, radical).
612 OTHER_CURVES = [
613     dict(name="y = 0.5x2 - 2x + 1", kind="parabola",
614           f=lambda x: 0.5 * x ** 2 - 2 * x + 1, abc=(0.5, -2, 1), pt=(5, -2),
615           bracket=(-10, 10), domain=None, xr=(-1, 7)),
616     dict(name="y = -x2 + 4x - 1", kind="parabola",
617           f=lambda x: -x ** 2 + 4 * x - 1, abc=(-1, 4, -1), pt=(-1, 4),
618           bracket=(-10, 10), domain=None, xr=(-2.5, 4.5)),
619     dict(name="y = e", kind="exponential",
620           f=np.exp, pt=(2, 1), bracket=(-5, 3), domain=None, xr=(-2.5, 3)),
621     dict(name="y = ln x", kind="logarithmic",
622           f=np.log, pt=(0, 1), bracket=(1e-3, 6), domain=(1e-6, 1e6), xr=(0.02, 5)),
623     dict(name="y = √x", kind="radical",
624           f=np.sqrt, pt=(4, 0), bracket=(0, 10), domain=(1e-9, 1e6), xr=(0, 6)),
625     dict(name="y = 1/x", kind="rational (x in denominator)",
626           f=lambda x: 1 / x, pt=(0, 0), bracket=(0.05, 8), domain=(1e-3, 1e6), xr=(0.15, 4)),
627     dict(name="y = 4/(1 + x2)", kind="rational (x in denominator)",
628           f=lambda x: 4 / (1 + x ** 2), pt=(3, 3), bracket=(-2, 8), domain=None, xr=(-3, 5)),
629     dict(name="y = x(1/3)", kind="cube root",
630           f=np.cbrt, pt=(-1, 2), bracket=(-6, 6), domain=None, xr=(-4, 3)),
631 ]
632
633
634 def other_curves_demo():
635     """The same two solvers on other parabolas and on non-polynomial curves.
636
637     Each answer is checked against a brute-force grid search, and the parabolas
638     are additionally checked against the exact cubic. Two 2x2 figures are drawn
639     (upright, so the labels stay readable).
640     """
641     print("\n" + "=" * 78)
642     print("PART 1 --- other curves and other points")
643     print("=" * 78)
644     print(f"{'function':>18} {'point':>9} | {'x* (Newton)':>12} {'d (Newton)':>11} {'it':>3} | "
645           f"{'d (golden)':>11} | {'d (grid)':>10} | {'d (exact)':>10}")
646     print("-" * 100)
647
648     figA, axsA = plt.subplots(2, 2, figsize=(10, 9))
649     figB, axsB = plt.subplots(2, 2, figsize=(10, 9))
650     axes = list(axsA.ravel()) + list(axsB.ravel())
651
652     rows = []
653     for g, ax in zip(OTHER_CURVES, axes):
654         f = g["f"]
655         x0, y0 = g["pt"]
656         r = distance_to_curve(x0, y0, f, g["bracket"], domain=g["domain"])
657         dn, xn, it = r["newton"]
658
659         exact = "----"
660         if g["kind"] == "parabola":
661             da, _, _ = analytical_parabola(x0, y0, *g["abc"])
662             exact = f"{da:10.6f}"
663         print(f"{g['name']:>18} {str(g['pt']):>9} | {xn:12.6f} {dn:11.6f} {it:3d} | "
664               f"{r['golden']>11.6f} | {r['grid']>10.6f} | {exact:>10}")
665         rows.append(dict(name=g["name"], kind=g["kind"], point=[x0, y0], **r))
666
667     # --- one panel per curve: the curve, the Newton iterates, the segment
668     xx = np.linspace(*g["xr"], 500)
669     ax.plot(xx, f(xx), color=BLUE, lw=2.4)
670     iterates = [h["x"] for h in r["newton_history"]] + [xn]
671     for i, xi in enumerate(iterates):
672         ax.scatter([xi], [f(xi)], s=26, color=ITER_CMAP(i / max(len(iterates) - 1, 1)),
673                   zorder=5, edgecolor=SURFACE, linewidth=0.8)
674     draw_shortest_segment(ax, x0, y0, xn, f)
675     draw_query_point(ax, x0, y0)
676

```

```

677     xa_, xb_ = min(g["xr"][0], x0 - 0.5), max(g["xr"][1], x0 + 0.5)
678     yv = np.concatenate([f(xx), [y0]])
679     ax.set_xlim(xa_, xb_); ax.set_ylim(yv.min() - 0.6, yv.max() + 0.6)
680     ax.set_aspect("equal", adjustable="datalim")
681     ax.set_title(g["name"], fontsize=12.5)
682     right = g["name"] in ("y = ln x", "y = -x2 + 4x -1")
683     ax.text(0.97 if right else 0.03, 0.04,
684             f"point ({x0}, {y0})\nd = {dn:.5f}\nx* = {xn:.5f}",
685             ha="right" if right else "left", transform=ax.transAxes, fontsize=9.3,
686             color=INK, bbox=dict(boxstyle="round,pad=0.35", fc="white", ec=GRID))
687
688     for fg, nm, sub in ((figA, "p1_gallery_a.png", "other parabolas, exponential, logarithm"),
689                       (figB, "p1_gallery_b.png", "radical, rational and cube-root functions")):
690         fg.suptitle(f"Same code, {sub} -Newton iterates (light →dark) and the shortest segment",
691                    x=0.01, ha="left", fontsize=13, fontweight="bold", color=INK)
692         fg.tight_layout()
693         save(fg, nm)
694     return rows
695
696
697 def multiple_minima_demo():
698     """A case where D has several stationary points: the point (0, 8).
699
700     D'(x) = 2x(2x2 - 5) has roots 0 and ±sqrt(2.5). x = 0 is a local *maximum*
701     of D, so Newton started at 0 stops on the wrong answer, while the analytical
702     solver enumerates all three roots and keeps the smallest D. This is why
703     newton_multistart exists.
704     """
705     x0, y0 = 0, 8
706     a, b, c = 1.0, 0.0, 5.0
707     f, df, ddf = parabola(a, b, c)
708     D = lambda x: dist_sq(x, x0, y0, f)
709
710     da, xa, roots = analytical_parabola(x0, y0, a, b, c)
711     starts = [-2.5, -0.4, 0.0, 0.4, 2.5]
712     runs = [newton_distance(x0, y0, f, df, ddf, initial_guess=s) for s in starts]
713     dg, xg, _ = golden_section(x0, y0, f, *BRACKET)
714
715     print("\n" + "=" * 78)
716     print("PART 1 --- several stationary points: the point (0, 8)")
717     print("=" * 78)
718     print("stationary points:", ", ".join(f"{r:+.6f}" for r in roots))
719     print(f"analytical: d = {da:.6f} at x* = {xa:.6f}")
720     for s, (d, x, _) in zip(starts, runs):
721         flag = " <-- local maximum, wrong answer" if abs(x) < 1e-9 else ""
722         print(f"Newton from x = {s:+.1f}: d = {d:.6f} at x* = {x:+.6f}{flag}")
723     print(f"golden section on [-10, 10]: d = {dg:.6f} at x* = {xg:+.6f}")
724
725     fig, axs = plt.subplots(1, 2, figsize=(12.5, 4.6))
726     ax = axs[0]
727     xx = np.linspace(-3.2, 3.2, 400)
728     ax.plot(xx, f(xx), color=BLUE, lw=2.5, label="y = x2 + 5")
729     for rt in roots:
730         is_min = abs(D(rt) - da * 2) < 1e-9
731         ax.plot([x0, rt], [y0, f(rt)], color=ORANGE if is_min else MUTED,
732                lw=1.8, ls=(0, (4, 2)))
733         ax.scatter([rt], [f(rt)], s=60, color=AQUA if is_min else MUTED,
734                   edgecolor=SURFACE, linewidth=2, zorder=6)
735     draw_query_point(ax, x0, y0, "(0, 8)", dx=0.2, dy=0.5)
736     ax.annotate("x = 0 is a critical point too\n(local MAX of D, d = 3)", (0, 5),
737                xytext=(0.5, 2.3), fontsize=9, color=INK2,
738                arrowprops=dict(arrowstyle="→", color=MUTED))
739     ax.set_aspect("equal"); ax.set_ylim(1.5, 11)
740     ax.set_xlabel("x"); ax.set_ylabel("y")
741     ax.set_title("(a) Two equally close points")
742
743     ax = axs[1]
744     xx = np.linspace(-2.6, 2.6, 400)

```

```

745 ax.plot(xx, D(xx), color=VIOLET, lw=2.3, label="D(x) = x2 + (x2 -3)2")
746 for s, (_, x, _) in zip(starts, runs):
747     ax.annotate("", xy=(x, D(x)), xytext=(s, D(s)),
748                 arrowprops=dict(arrowstyle="->", color=ORANGE, lw=1.4,
749                                 connectionstyle="arc3,rad=-0.3"))
750     ax.scatter([s], [D(s)], s=30, color=ORANGE, zorder=5)
751 for rt in roots:
752     ax.scatter([rt], [D(rt)], s=70, color=AQUA if D(rt) < 8 else RED,
753               edgecolor=SURFACE, linewidth=2, zorder=6)
754 ax.set_xlabel("x"); ax.set_ylabel("D(x)")
755 ax.set_title("(b) Newton from 5 starting guesses")
756 ax.text(0, 10.2, "x = 0 → stuck at\nthe local max", ha="center", fontsize=9, color=INK2)
757 ax.legend(loc="upper center", fontsize=9, bbox_to_anchor=(0.5, 0.82))
758 fig.tight_layout()
759 save(fig, "p1_multimin.png")
760
761 return dict(point=[x0, y0], roots=roots, d=da, x=xa,
762            newton=[dict(start=s, d=r[0], x=r[1]) for s, r in zip(starts, runs)],
763            golden=dict(d=dg, x=xg))
764
765
766 def main():
767     assigned_points_demo()
768     other_curves_demo()
769     multiple_minima_demo()
770     print("\nAll Part 1 figures written to ../media/\n")
771
772
773 if __name__ == "__main__":
774     main()

```

C fitting.py — Part 2 code

```

1  """
2  SYDE 572 - Assignment 1, Part 2
3  =====
4  Least-squares fitting of a line  $y = m x + b$  and a parabola  $y = a x^2 + b x + c$ 
5  to the points (0, 0.5), (2, 3.5), (1, 1.5), (3, 7.5), by minimising
6
7       $MSE(\theta) = (1/n) * \sum_i (g(x_i; \theta) - y_i)^2$  .
8
9  Run this file to reproduce every Part 2 result:
10
11      python fitting.py
12
13  It prints the parameter/MSE tables and writes the Part 2 figures to ../media/.
14
15  Contents
16  -----
17  Section 1 The data, the design matrix, the MSE and its derivatives
18  Section 2 Analytical solution: the normal equations
19  Section 3 Numerical solution: Newton-Raphson, one parameter per step
20  Section 4 Numerical solution: the full multivariate Newton step (comparison)
21  Section 5 Plots of the intermediate steps
22  Section 6 Tables and the demonstration run
23
24  Both models are linear in their parameters, so writing them as
25   $y_{\text{hat}} = \Phi @ \theta$  lets one gradient and one Hessian serve both (and any
26  polynomial degree). numpy is used only for array arithmetic; the linear solve
27  is a hand-written Gaussian elimination.
28  """
29
30  import os
31
32  import matplotlib
33  matplotlib.use("Agg") # write PNG files, no interactive window
34  import matplotlib.pyplot as plt
35  import numpy as np
36  from matplotlib.colors import LinearSegmentedColormap
37
38  MEDIA = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "media")
39
40  # =====
41  # Section 1. Data, model, and the MSE
42  # =====
43  # The assigned data set (order is irrelevant to the fit).
44  X = np.array([0.0, 2.0, 1.0, 3.0])
45  Y = np.array([0.5, 3.5, 1.5, 7.5])
46
47
48  def design(x, degree):
49      """Design matrix Phi with columns  $x^{\text{degree}}$ , ...,  $x$ , 1.
50
51      degree 1 ->  $[x, 1]$  and  $\theta = [m, b]$  (the line)
52      degree 2 ->  $[x^2, x, 1]$  and  $\theta = [a, b, c]$  (the parabola)
53
54      With this, the model is simply  $y_{\text{hat}} = \Phi @ \theta$  for both fits.
55      """
56      return np.vstack([x ** p for p in range(degree, -1, -1)]).T
57
58
59  def mse(theta, x=X, y=Y):
60      """Mean squared error of the model with parameters theta."""
61      residuals = design(x, len(theta) - 1) @ theta - y
62      return float(np.mean(residuals ** 2))
63
64

```

```

65 def gradient(theta, x=X, y=Y):
66     """d(MSE)/d(theta) = (2/n) Phi^T (Phi theta - y).
67
68     For the line this is exactly the pair
69         dMSE/dm = (14m + 6b - 31)/2, dMSE/db = (6m + 4b - 13)/2
70     obtained by hand from the sums of x_i, x_i^2, y_i, x_i y_i.
71     """
72     P = design(x, len(theta) - 1)
73     return 2 / len(x) * P.T @ (P @ theta - y)
74
75
76 def hessian(theta, x=X):
77     """d^2(MSE)/d(theta)^2 = (2/n) Phi^T Phi.
78
79     It does not depend on theta, because the MSE is quadratic in the parameters.
80     The diagonal entries are the second derivatives used by the one-parameter
81     Newton steps; the off-diagonal entries are the cross terms that the
82     one-parameter scheme ignores (and the full Newton step uses).
83     """
84     P = design(x, len(theta) - 1)
85     return 2 / len(x) * P.T @ P
86
87
88 # =====
89 # Section 2. Analytical solution: the normal equations
90 # =====
91 def gauss_solve(A, r):
92     """Solve A t = r by Gaussian elimination with partial pivoting.
93
94     Written out rather than calling numpy.linalg.solve, since the assignment
95     asks for the analytical procedure to be implemented.
96     """
97     A = np.array(A, float)
98     r = np.array(r, float)
99     n = len(r)
100
101     # Forward elimination: pivot on the largest entry for numerical stability.
102     for i in range(n):
103         p = i + np.argmax(abs(A[i:, i]))
104         A[[i, p]], r[[i, p]] = A[[p, i]], r[[p, i]]
105         for j in range(i + 1, n):
106             factor = A[j, i] / A[i, i]
107             A[j, i:] -= factor * A[i, i:]
108             r[j] -= factor * r[i]
109
110     # Back substitution.
111     t = np.zeros(n)
112     for i in range(n - 1, -1, -1):
113         t[i] = (r[i] - A[i, i + 1:] @ t[i + 1:]) / A[i, i]
114     return t
115
116
117 def normal_equations(degree, x=X, y=Y):
118     """Analytical least-squares fit: solve (Phi^T Phi) theta = Phi^T y.
119
120     Setting every partial derivative of the MSE to zero gives exactly this
121     linear system - the 2x2 system for the line and the 3x3 system for the
122     parabola that are solved by hand in the write-up.
123     """
124     P = design(x, degree)
125     return gauss_solve(P.T @ P, P.T @ y)
126
127
128 # =====
129 # Section 3. Numerical solution: Newton-Raphson, one parameter per step
130 # =====
131 def coordinate_newton(theta0, x=X, y=Y, tolerance=1e-10, max_iter=500):
132     """Optimise one parameter at a time, as the assignment specifies.

```

```

133
134     One outer iteration ("sweep") runs, for each parameter  $j$  in turn:
135
136          $\theta_j \leftarrow \theta_j - (d\text{MSE}/d\theta_j) / (d^2 \text{MSE}/d\theta_j^2)$ 
137
138     with every other parameter held at its latest value. Because the MSE is
139     quadratic in each parameter separately, this 1-D Newton step is exact: it
140     lands on that parameter's minimiser given the others. For the line the two
141     updates reduce to  $m \leftarrow (31 - 6b)/14$  and  $b \leftarrow (13 - 6m)/4$ .
142
143     Convergence is only linear, because the coordinate directions ignore the
144     cross term in the Hessian (see full_newton for the contrast).
145
146     Returns  $(\theta, \text{history})$  where history records  $\theta$  and the MSE after every
147     single sub-step, which is what the intermediate-step plots use.
148     """
149     theta = np.array(theta0, float)
150     history = [dict(iter=0, step="start", theta=theta.copy(), mse=mse(theta, x, y))]
151
152     for it in range(1, max_iter + 1):
153         previous = theta.copy()
154         for j in range(len(theta)):
155             g = gradient(theta, x, y)[j] # first derivative w.r.t.  $\theta_j$ 
156             h = hessian(theta, x)[j, j] # second derivative w.r.t.  $\theta_j$ 
157             theta[j] -= g / h
158             history.append(dict(iter=it, step=j, theta=theta.copy(), mse=mse(theta, x, y)))
159             if np.max(abs(theta - previous)) < tolerance:
160                 break
161
162     return theta, history
163
164
165 # =====
166 # Section 4. Numerical solution: the full multivariate Newton step
167 # =====
168 def full_newton(theta0, x=X, y=Y, tolerance=1e-12, max_iter=50):
169     """Update all parameters together:  $\theta \leftarrow \theta - H^{-1} \text{grad}$ .
170
171     For a quadratic MSE the Hessian is constant and this reaches the exact
172     minimum in a single step from any starting point. Included to show what the
173     one-parameter-at-a-time scheme gives up.
174     """
175     theta = np.array(theta0, float)
176     history = [dict(iter=0, theta=theta.copy(), mse=mse(theta, x, y))]
177
178     for it in range(1, max_iter + 1):
179         step = gauss_solve(hessian(theta, x), gradient(theta, x, y))
180         theta = theta - step
181         history.append(dict(iter=it, theta=theta.copy(), mse=mse(theta, x, y)))
182         if np.max(abs(step)) < tolerance:
183             break
184
185     return theta, history
186
187
188 # =====
189 # Section 5. Plots
190 # =====
191 INK, INK2, MUTED, GRID = "#0b0b0b", "#52514e", "#8a8984", "#e6e5e0"
192 BLUE, ORANGE, AQUA, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7"
193 SURFACE = "#fcfcfb"
194 ITER_CMAP = LinearSegmentedColormap.from_list("iter", ["#b9d4f3", "#2a78d6", "#0f3c75"])
195
196 plt.rcParams.update({
197     "figure.dpi": 110, "savefig.dpi": 170,
198     "figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "savefig.facecolor": SURFACE,
199     "font.family": "DejaVu Sans", "font.size": 10.5,
200     "axes.edgecolor": MUTED, "axes.labelcolor": INK2, "axes.titlecolor": INK,

```

```

201 "axes.titlesize": 12, "axes.titleweight": "bold", "axes.titlelocation": "left",
202 "axes.titlepad": 10,
203 "axes.spines.top": False, "axes.spines.right": False,
204 "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.8,
205 "xtick.color": INK2, "ytick.color": INK2, "legend.frameon": False,
206 "lines.linewidth": 2, "mathtext.fontset": "dejavusans",
207 })
208
209
210 def save(fig, name):
211     os.makedirs(MEDIA, exist_ok=True)
212     fig.savefig(os.path.join(MEDIA, name), bbox_inches="tight", pad_inches=0.15)
213     plt.close(fig)
214     print(" figure:", name)
215
216
217 def plot_fits(line, parab, name="p2_fits.png"):
218     """The two fitted curves with their residuals, and a residual bar chart."""
219     fig, axs = plt.subplots(1, 2, figsize=(13, 4.8), gridspec_kw=dict(width_ratios=[1.5, 1]))
220     xx = np.linspace(-0.3, 3.3, 300)
221
222     ax = axs[0]
223     ax.plot(xx, design(xx, 1) @ line, color=BLUE, lw=2.4,
224             label=f"line y = {line[0]:.2f}x {line[1]:+.2f} (MSE = {mse(line):.4f})")
225     ax.plot(xx, design(xx, 2) @ parab, color=ORANGE, lw=2.4,
226             label=f"parabola y = {parab[0]:.2f}x2 {parab[1]:+.2f}x {parab[2]:+.2f}"
227                 f" (MSE = {mse(parab):.4f})")
228     # residual whiskers, one per data point and model
229     for xi, yi in zip(X, Y):
230         ax.plot([xi, xi], [yi, design(np.array([xi]), 1)[0] @ line], color=BLUE, lw=1, alpha=0.5)
231         ax.plot([xi + 0.03] * 2, [yi, design(np.array([xi]), 2)[0] @ parab],
232                 color=ORANGE, lw=1, alpha=0.6)
233     ax.scatter(X, Y, s=85, color=INK, edgecolor=SURFACE, linewidth=2, zorder=6, label="data")
234     for xi, yi in zip(X, Y):
235         ax.annotate(f"({xi:g}, {yi:g})", (xi, yi), xytext=(-12, 10),
236                    textcoords="offset points", fontsize=9.5, color=INK2, ha="right")
237     ax.set_xlabel("x"); ax.set_ylabel("y")
238     ax.set_title("Least-squares fits (thin lines = residuals)")
239     ax.legend(loc="upper left", fontsize=9.5)
240
241     ax = axs[1]
242     order = np.argsort(X)
243     w = 0.36
244     ax.bar(np.arange(4) - w / 2 - 0.01, (design(X, 1) @ line - Y)[order], w,
245            color=BLUE, label="line")
246     ax.bar(np.arange(4) + w / 2 + 0.01, (design(X, 2) @ parab - Y)[order], w,
247            color=ORANGE, label="parabola")
248     ax.axhline(0, color=INK2, lw=1)
249     ax.set_xticks(range(4), [f"x = {v:g}" for v in X[order]])
250     ax.set_ylabel("residual  $\hat{y}-y$ ")
251     ax.set_title("Residuals")
252     ax.legend(fontsize=9.5)
253
254     fig.tight_layout()
255     save(fig, name)
256
257
258 def plot_line_path(line, coord_hist, full_hist, name="p2_line_path.png"):
259     """Intermediate steps for the line fit.
260
261     (a) contours of  $MSE(m, b)$  with the coordinate-wise path (axis-aligned
262         zig-zag, magnified in an inset) and the single full Newton step
263     (b) the fitted line after each sweep
264     """
265     fig, axs = plt.subplots(1, 2, figsize=(13.5, 5.2))
266
267     # ---- (a) contour plot of the objective -----
268     ax = axs[0]

```

```

269 mm, bb = np.meshgrid(np.linspace(-0.3, 3.3, 300), np.linspace(-2.2, 1.6, 300))
270 Z = np.mean((mm[... , None] * X + bb[... , None] - Y) ** 2, axis=-1)
271 cs = ax.contour(mm, bb, Z, levels=[0.6, 0.75, 1, 1.5, 2.5, 4, 6, 9, 13, 18, 25],
272               cmap=LinearSegmentedColormap.from_list("c", ["#9085e9", "#d6d2f5"]),
273               linewidths=1)
274 ax.clabel(cs, fontsize=7.5, fmt="%g")
275
276 P = np.array([e["theta"] for e in coord_hist[:1 + 2 * 15]])
277 ax.plot(P[:, 0], P[:, 1], color=ORANGE, lw=1.6, marker="o", ms=4.5, mec=SURFACE,
278         label="coordinate Newton (m-step, then b-step)")
279 ax.annotate("start (0, 0)", P[0], xytext=(6, 6), textcoords="offset points",
280           fontsize=9, color=INK)
281
282 # inset: the steps are tiny next to the full contour range
283 ins = ax.inset_axes([0.06, 0.40, 0.36, 0.36])
284 ins.contour(mm, bb, Z, levels=[0.576, 0.58, 0.585, 0.6, 0.65, 0.75],
285           colors="#b8b0f0", linewidths=0.8)
286 ins.plot(P[:, 0], P[:, 1], color=ORANGE, lw=1.3, marker="o", ms=3.5, mec=SURFACE)
287 for i, lab in ((1, "m"), (2, "b"), (3, "m"), (4, "b")):
288     ins.annotate(lab, P[i], xytext=(4, 3), textcoords="offset points",
289               fontsize=8.5, color=INK)
290 ins.scatter([line[0]], [line[1]], s=90, marker="*", color=AQUA, edgecolor=INK, zorder=7)
291 ins.set_xlim(2.18, 2.33); ins.set_ylim(-0.24, 0.03)
292 ins.set_title("zoom: axis-aligned m-step / b-step zig-zag", fontsize=8.5,
293             fontweight="normal")
294 ins.tick_params(labelsize=7.5); ins.grid(False)
295 ax.indicate_inset_zoom(ins, edgecolor=MUTED)
296
297 F = np.array([e["theta"] for e in full_hist])
298 ax.plot(F[:, 2, 0], F[:, 2, 1], color=BLUE, lw=1.6, ls="--", label="full Newton (one step)")
299 ax.scatter([line[0]], [line[1]], s=110, marker="*", color=AQUA, edgecolor=INK, zorder=7,
300           label=f"optimum m = {line[0]:.2f}, b = {line[1]:.2f}")
301 ax.set_xlabel("slope m"); ax.set_ylabel("intercept b")
302 ax.set_title("(a) MSE(m, b) contours and the optimisation path")
303 ax.legend(loc="lower left", fontsize=9)
304 ax.grid(False)
305
306 # ---- (b) the fitted line after each sweep -----
307 ax = axs[1]
308 xx = np.linspace(-0.3, 3.3, 100)
309 sweeps = [e for e in coord_hist if e["step"] in ("start", 1)][1:]
310 for i, e in enumerate(sweeps):
311     ax.plot(xx, design(xx, 1) @ e["theta"], color=ITER_CMAP(i / (len(sweeps) - 1)), lw=1.8,
312           label=f"sweep {e['iter']}: y = {e['theta'][0]:.3f}x {e['theta'][1]:+.3f}"
313             f" (MSE {e['mse']:.3f})")
314 ax.scatter(X, Y, s=70, color=INK, edgecolor=SURFACE, linewidth=2, zorder=6)
315 ax.set_xlabel("x"); ax.set_ylabel("y")
316 ax.set_title("(b) Intermediate lines after each sweep")
317 ax.legend(fontsize=8.3, loc="upper left")
318
319 fig.tight_layout()
320 save(fig, name)
321
322
323 def plot_parabola_and_convergence(line, parab, line_hist, parab_hist,
324                                line_full, parab_full, name="p2_convergence.png"):
325     """(a) the parabola after selected sweeps; (b) excess MSE per sweep, log scale."""
326     fig, axs = plt.subplots(1, 2, figsize=(13.5, 5.0))
327     xx = np.linspace(-0.3, 3.3, 100)
328
329     ax = axs[0]
330     show = [0, 1, 2, 3, 5, 10, 30]
331     by_sweep = {e["iter"]: e for e in parab_hist if e["step"] in ("start", 2)}
332     for i, s in enumerate(show):
333         e = by_sweep[s]
334         a_, b_, c_ = e["theta"]
335         ax.plot(xx, design(xx, 2) @ e["theta"], color=ITER_CMAP(i / (len(show) - 1)), lw=1.8,
336               label=f"sweep {s}: {a_:.3f}x² {b_:.3f}x {c_:.3f} (MSE {e['mse']:.4f})")

```

```

337 ax.plot(xx, design(xx, 2) @ parab, color=ORANGE, lw=2.4, ls="--", label="analytical optimum")
338 ax.scatter(X, Y, s=70, color=INK, edgecolor=SURFACE, linewidth=2, zorder=6)
339 ax.set_xlabel("x"); ax.set_ylabel("y")
340 ax.set_title("(a) Parabola after each coordinate sweep")
341 ax.legend(fontsize=8, loc="upper left")
342
343 ax = axs[1]
344 # Excess MSE above the optimum; floored so the log axis stays readable once
345 # the iteration reaches floating-point noise.
346 lm = [e["mse"] - mse(line) for e in line_hist if e["step"] in ("start", 1)]
347 pm = [e["mse"] - mse(parab) for e in parab_hist if e["step"] in ("start", 2)]
348 ax.semilogy(range(len(lm)), np.maximum(lm, 1e-15), color=BLUE, lw=2,
349             label="line (2 parameters)")
350 ax.semilogy(range(len(pm)), np.maximum(pm, 1e-15), color=ORANGE, lw=2,
351             label="parabola (3 parameters)")
352 ax.scatter([1, 1], [max(line_full[1]["mse"] - mse(line), 1e-15),
353                    max(parab_full[1]["mse"] - mse(parab), 1e-15)],
354            marker="*", s=120, color=AQUA, edgecolor=INK, zorder=6,
355            label="full Newton after 1 step")
356 ax.set_xlim(-2, 150)
357 ax.set_xlabel("sweep (one Newton step per parameter)")
358 ax.set_ylabel("MSE -MSE* (floored at 1e-15)")
359 ax.set_title("(b) Convergence of coordinate-wise Newton")
360 ax.legend(fontsize=9)
361
362 fig.tight_layout()
363 save(fig, name)
364
365
366 # =====
367 # Section 6. Tables and the demonstration run
368 # =====
369 def print_sweep_table(history, names, nsweeps):
370     """Print theta and the MSE after every sub-step of the first few sweeps."""
371     header = " ".join(f"{n:>10}" for n in names)
372     print(f"{'sweep':>5} {'updated':>8} {header} {'MSE':>10}")
373     for e in history[:1 + len(names) * nsweeps]:
374         step = "start" if e["step"] == "start" else names[e["step"]]
375         vals = " ".join(f"{v:10.6f}" for v in e["theta"])
376         print(f"{'iter':>5d} {step:>8} {vals} {e['mse']:10.6f}")
377
378
379 def main():
380     print("\n" + "=" * 78)
381     print("PART 2 --- fitting a line and a parabola by minimising the MSE")
382     print("=" * 78)
383     print("data:", " ".join(f"{x:g}, {y:g}" for x, y in zip(X, Y)))
384
385     results = {}
386     for label, degree, names in (("line", 1, ["m", "b"]),
387                                ("parabola", 2, ["a", "b", "c"])):
388         analytic = normal_equations(degree)
389         coord, coord_hist = coordinate_newton(np.zeros(degree + 1))
390         full, full_hist = full_newton(np.zeros(degree + 1))
391
392         print(f"\n{label.upper()} ({', '.join(names)})")
393         print(f"analytical (normal equations) : "
394               f"{'', '.join(f'{n}={v:.6f}' for n, v in zip(names, analytic))} "
395               f"MSE = {mse(analytic):.6f}")
396         print(f"Newton, one parameter at a time: "
397               f"{'', '.join(f'{n}={v:.6f}' for n, v in zip(names, coord))} "
398               f"MSE = {mse(coord):.6f} ({coord_hist[-1]['iter']} sweeps)")
399         print(f"Newton, full multivariate step : "
400               f"{'', '.join(f'{n}={v:.6f}' for n, v in zip(names, full))} "
401               f"MSE = {mse(full):.6f} (1 step)")
402
403     residuals = design(X, degree) @ analytic - Y
404     print("residuals (y_hat - y):",

```

```

405         ", ".join(f"x={x:g}: {r:+.4f}" for x, r in zip(X, residuals)))
406     print(f"\n first sweeps of the one-parameter-at-a-time iteration:")
407     print_sweep_table(coord_hist, names, 6 if degree == 1 else 4)
408
409     results[label] = dict(analytic=analytic, coord=coord, coord_hist=coord_hist,
410                          full=full, full_hist=full_hist)
411
412     line, parab = results["line"], results["parabola"]
413     plot_fits(line["analytic"], parab["analytic"])
414     plot_line_path(line["analytic"], line["coord_hist"], line["full_hist"])
415     plot_parabola_and_convergence(line["analytic"], parab["analytic"],
416                                  line["coord_hist"], parab["coord_hist"],
417                                  line["full_hist"], parab["full_hist"])
418
419     print(f"\n ANSWERS: line y = {line['analytic'][0]:.2f}x {line['analytic'][1]:+.2f}"
420           f" MSE = {mse(line['analytic']):.4f}")
421     print(f" parabola y = {parab['analytic'][0]:.2f}x^2 "
422           f"{parab['analytic'][1]:+.2f}x {parab['analytic'][2]:+.2f}"
423           f" MSE = {mse(parab['analytic']):.4f}")
424     print("\nAll Part 2 figures written to ../media/\n")
425
426
427 if __name__ == "__main__":
428     main()

```